

Symmetry-Augmented Multi-Agent Reinforcement Learning for Scalable UAV Trajectory Design and User Scheduling

Xuanhan Zhou¹, Jun Xiong¹, Haitao Zhao¹, *Senior Member, IEEE*, Chao Yan², and Jibo Wei², *Member, IEEE*

Abstract—Unmanned aerial vehicles (UAVs) as mobile base stations are recognized as effective means for emergency communications. The performance of such systems depends on the movement of UAVs and scheduling of ground users (GUs). However, devising an efficient algorithm to jointly optimize UAV trajectories and user scheduling is still challenging, especially in real-time scenarios lacking central controllers. Multi-agent deep reinforcement learning (MADRL) provides a promising solution to this problem. Nevertheless, as the numbers of UAVs and GUs increase, existing MADRL algorithms encounter scalability and sample efficiency issues. In this paper, we develop a novel symmetry-augmented MADRL approach for learning scalable UAV trajectory design and user scheduling policies. The core idea is to utilize symmetries to reduce the multi-agent state-action space and enhance sample efficiency. Specifically, we design a family of neural networks to learn individual policies, namely entity permutation equivariant policy networks (EP2Nets). EP2Nets effectively leverage the permutation symmetry to reduce redundancy in the state-action space. Additionally, we achieve data augmentation by exploiting rotational and reflection symmetries, further boosting sample efficiency. Finally, a Symmetric QMIX (SymmQMIX) algorithm is proposed by integrating the EP2Net and data augmentation method into the QMIX algorithm. Simulation results indicate that SymmQMIX significantly outperforms QMIX and other symmetry-enhanced algorithms, achieving a 4.5-fold increase in converged performance and a 100-fold improvement in sample efficiency.

Index Terms—Multi-agent deep reinforcement learning (MADRL), symmetry, trajectory design, unmanned aerial vehicle (UAV).

I. INTRODUCTION

UNMANNED aerial vehicles (UAVs) as mobile base stations (BSs) have emerged as an effective solution for

emergency communications [1]. Compared with terrestrial infrastructures, UAVs can be rapidly deployed to any location. Besides, their exceptional maneuverability enables real-time adjustments in their positions to accommodate evolving communication needs [2]. However, individual UAVs have limited power and communication capacity. To overcome these limitations, deploying multiple UAVs to cooperatively serve ground users (GUs) has become a common strategy to extend the network coverage and enhance overall communication capability [3].

In multi-UAV assisted communications, the quality of service received by GUs depends on the positions of UAVs and the communication scheduling of GUs, which are closely coupled [4]. Therefore, the joint optimization of UAV trajectories and user scheduling can significantly enhance system performance. However, this problem exhibits high complexity, especially in scenarios involving numerous UAVs and GUs. Traditional optimization-based algorithms pre-design UAV trajectories and communication parameters offline, relying on static models and prior information [5], [6], [7], [8], [9]. They encounter difficulties in adapting to unexpected environmental variations [10]. Moreover, these methods necessitate centralized control and global information, rendering them impractical in multi-UAV systems lacking central controllers [11].

Multi-agent deep reinforcement learning (MADRL) [12], [13], [14], [15] offers a promising solution to the above challenges. In MADRL methods, UAVs are treated as individual agents, and a policy network is trained for each agent using experience gathered through their interactions with the environment [16]. After sufficient training, these policy networks can be utilized to make UAV trajectory and communication decisions in real time. This approach not only facilitates distributed and online decision-making but also promotes cooperation among UAVs.

Great efforts have been devoted to developing MADRL algorithms for the joint UAV trajectory and communication optimization problems [17], [18], [19], [20]. While these algorithms demonstrate satisfactory performance in small-scale problems, they encounter scalability and sample efficiency issues [21]. In the joint UAV trajectory design and user scheduling problem, the combined actions associated with trajectories and scheduling contribute to a substantial action space. Furthermore, both state and action dimensions scale with the increase in the number of UAVs and GUs, resulting in an explosive expansion of the state-action space. Consequently, existing MADRL algorithms

Manuscript received 25 December 2023; revised 29 May 2024; accepted 30 July 2024. Date of publication 9 August 2024; date of current version 5 November 2024. This work was supported in part by the China Natural Science Foundation under Grant 62371462, under Grant 61931020, under Grant 62101569, under Grant 62171449, and under Grant U19B2024, in part by Natural Science Foundation of Hunan Province under Grant 2022JJ10068 and in part by the Science and Technology Innovation Program of Hunan Province under Grant 2022RC1093. Recommended for acceptance by D.T. Hoang. (*Corresponding author: Jun Xiong.*)

Xuanhan Zhou, Jun Xiong, Haitao Zhao, and Jibo Wei are with the College of Electronic Science and Technology, National University of Defense Technology, Changsha, Hunan 410073, China (e-mail: zhouxuanhan15@nudt.edu.cn; xj8765@nudt.edu.cn; haitaozhao@nudt.edu.cn; wjbhw@nudt.edu.cn).

Chao Yan is with the College of Automation Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing, Jiangsu 211106, China (e-mail: yanchao@nuaa.edu.cn).

Digital Object Identifier 10.1109/TMC.2024.3437679

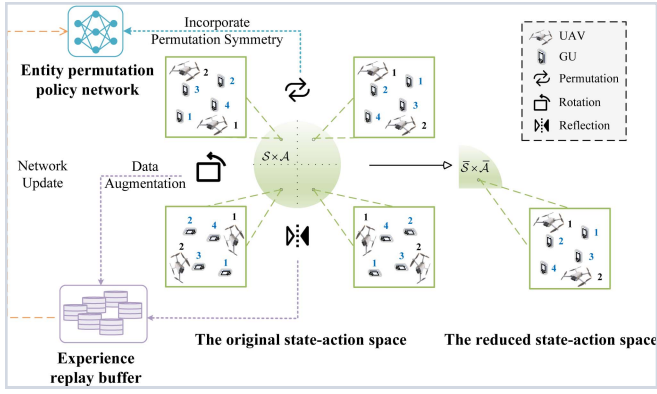


Fig. 1. An illustration of the proposed symmetry-augmented MADRL approach. The permutation symmetry is incorporated into entity permutation policy networks to remove redundancy caused by different entity permutations. Additionally, the rotational and reflection symmetries are exploited to augment experience samples for more sample-efficient network updates.

require a substantial number of experience samples to achieve convergence in large scale scenarios. This renders them susceptible to getting trapped in local optima, leading to deterioration of converged performance [22].

Multi-UAV assisted communication systems possess various symmetries that can be exploited to reduce the state-action space. In this context, symmetries refer to transformations of state-action pairs that leave the rewards and state transition probabilities invariant [23]. Fig. 1 illustrates three symmetries inherent in multi-UAV assisted communications: permutation symmetry, rotational symmetry, and reflection symmetry. These symmetries establish equivalence between the original state-action pairs and their transformed counterparts, enabling experience gained from one pair to improve the policy for all equivalent pairs. By leveraging these symmetries, we can remove redundancy in the state-action space, thereby enhancing sample efficiency and converged performance.

The effectiveness of leveraging symmetries has been demonstrated in deep reinforcement learning (DRL), either through encoding symmetries into policy networks [24], [25], [26], or by employing data augmentation to expand experience samples [27]. These approaches rely on global symmetries in the joint state-action space. This limitation restricts their applicability in MADRL, where symmetries exist at both the global environment and individual agent levels [28]. A few studies have explored the utilization of permutation symmetry in MADRL [29], [30], [31], [32]. However, beyond permutation symmetry, multiple other symmetries exist in multi-UAV assisted communications, including rotational and reflection symmetries. Therefore, a more comprehensive MADRL approach is necessary to leverage diverse symmetries.

In this paper, we propose a novel symmetry-augmented MADRL approach to solve the UAV trajectory and user scheduling optimization problem. As illustrated in Fig. 1, the core idea of our approach is to incorporate the permutation symmetry into policy networks, and exploit the rotational and reflection symmetries to augment experience samples. These techniques result in a substantial reduction of the state-action space, thereby

improving sample efficiency and scalability. To the best of our knowledge, this marks the initial endeavor to harness symmetries in MADRL-based UAV trajectory and communication optimization problems. The main contributions are summarized as follows:

- We convert the problem into a decentralized partially observable Markov decision process (Dec-POMDP), where states, observations, and actions are represented in an entity-factored manner. This formulation allows us to transform the global symmetries into distributed symmetries at the agent level, laying the foundation for subsequent method design.
- We design a family of neural networks for learning decentralized UAV trajectory design and user scheduling policies, namely entity permutation equivariant policy networks (EP2Nets). EP2Nets effectively reduce redundancy in the state-action space by utilizing the permutation symmetry, thereby improving sample efficiency and algorithm performance.
- We achieve data augmentation by leveraging rotational and reflection symmetries. This method facilitates efficient learning from limited experience samples, reducing the need for costly agents-environment interactions.
- We propose a Symmetric QMIX (SymmQMIX) algorithm by integrating the EP2Net and data augmentation method into the QMIX algorithm. Extensive simulations demonstrate that SymmQMIX significantly outperforms QMIX and other symmetry-enhanced algorithms [12] in terms of sample efficiency and converged performance.

The rest of this paper is organized as follows. Section II reviews existing research on multi-UAV assisted communications and related MADRL techniques. Section III-A defines the system model and formulates the joint UAV trajectory and user scheduling optimization problem. Section IV converts the problem into a Dec-POMDP and formulates the mathematical model for symmetries. Section V presents the details of the proposed approach. Section VI tests the effectiveness of the proposed approach through extensive simulations. Finally, Section VII concludes this work.

II. RELATED WORK

Early research on multi-UAV assisted communications primarily solves UAV trajectory and communication problems using optimization-based algorithms. In [5], Wu et al. jointly optimize UAV trajectories, user scheduling, and power control to maximize the minimum throughput of all GUs. They employ block coordinate descent (BCD) [33] and successive convex optimization (SCA) [34] techniques to solve this nonconvex problem. BCD and SCA have also been successfully applied in other contexts, such as UAV-aided data collection [6] and UAV-assisted mobile edge computing (MEC) [7]. To reduce the computational complexity of the joint UAV trajectory and power control problem, Shen et al. [8] develop a parallel algorithm utilizing alternating direction method of multipliers technique. In [9], the UAV placement and movement optimization problem is studied for the multi-UAV coordinated multipoint

communications. These studies optimize UAV trajectories and communication variables offline, restricting their applicability in dynamic environments. Moreover, global information and centralized control are required in these approaches, which are often unavailable in multi-UAV systems.

MADRL is considered as a promising technology for real-time applications, especially in situations when central controllers are absent. In [17] and [18], independent deep Q-network (IDQN) algorithm [35] is adopted to train policies for UAV agents. These policies are then utilized for online trajectory control. Given predefined UAV trajectories, Yuan et al. [19] employ actor-critic algorithm [36] to train UAVs' user scheduling policies. Yin et al. [37] apply the QMIX algorithm [12] to optimize UAVs' trajectory design and user scheduling policies. Ding et al. [20] model UAVs and GUs as heterogeneous agents, and learn their policies using multi-agent deep deterministic policy gradient (MADDPG) algorithm [14]. These approaches provide various real-time and decentralized solutions, demonstrating satisfactory performance in small-scale problems. However, as the number of UAVs and GUs increases, the state-action space grows explosively, resulting in poor sample efficiency and scalability of these methods.

Recent research has demonstrated the potential of leveraging symmetries to improve sample efficiency in DRL problems, mainly through policy network design and data augmentation. In [23], Zinkevich et al. examine the notion of symmetries in Markov decision processes (MDPs). Clark et al. [24] encode symmetries related to playing Go into convolutional neural networks (CNNs). Simm et al. [25] propose a covariant actor-critic architecture for designing 3D symmetric molecules. Vanderpol et al. [26] introduce MDP homomorphic, a family of deep architectures that tie weights over symmetric state-action pairs. In [27], Lin et al. generate synthetic experience samples by applying symmetry transformations to real experience samples. These approaches depend on symmetries in the joint state-action space, and are not applicable in MADRL problems, where symmetries exist at both the global environment and individual agent levels.

In MADRL, research on symmetry is still relatively limited, but the application of permutation invariance [38] has begun to attract attention. In many multi-agent tasks, there is no specific order among entities. Changing the order of entity features in states and observations should maintain invariance of actions. Therefore, the policy network needs to satisfy permutation invariance, which is a special case of permutation symmetry. Efforts have been made to enhance sample efficiency of MADRL through the utilization of permutation invariance [29], [30], [31], [32]. In these studies, homogeneous entities are treated as anonymous, ensuring that permuting entity orders does not affect agents' policies. This is achieved by employing natural permutation-invariant architectures, such as DeepSets [38], Attention [39], and graph neural networks (GNNs) [40]. However, in the joint UAV trajectory design and user scheduling problem, the permutation of entity orders leads to synchronous changes in the input observations and output actions of the policy network, necessitating the consideration of more general permutation equivariance rather than invariance. Currently,

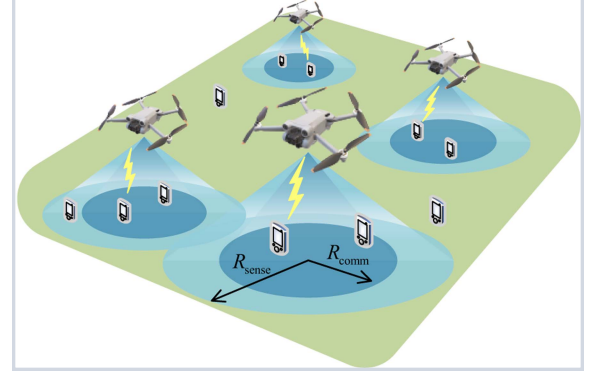


Fig. 2. An illustrative example of system model. Four UAVs are providing communication services for four GUs. The communication range and sensing range for each UAV are R_{comm} and R_{sense} , respectively.

permutation equivariance has received limited attention in the field of MADRL, with only preliminary exploration conducted by [31]. Moreover, these studies lack a comprehensive theoretical analysis for symmetries in MADRL, making it challenging to accommodate more types of symmetries.

In contrast to the aforementioned research, this paper establishes a theoretical foundation for symmetries in MADRL by extending the concepts of symmetries from DRL. Based on this foundation, we consider multiple symmetries inherent in multi-UAV assisted communications and incorporate them into both policy networks and the learning process. As far as we know, no prior work has investigated symmetries in the joint UAV trajectory and communication problems. This research gap serves as the primary motivation for our study.

III. SYSTEM MODEL AND PROBLEM FORMULATION

A. System Model

Fig. 2 illustrates a multi-UAV assisted communication system, where UAVs serve as mobile base stations to cooperatively provide communication services for GUs. Our objective is to improve the overall communication performance by jointly optimizing UAV trajectories and user scheduling.

We consider a square area of size $D \times D$. The sets of UAVs and GUs are denoted as $\mathcal{N} = \{1, \dots, N\}$ and $\mathcal{M} = \{1, \dots, M\}$, respectively. We divide the entire service duration into multiple equal time slots, each with a slot length τ . These time slots are represented as $\mathcal{T} = \{1, \dots, T\}$. UAVs maintain a constant speed V and flying altitude H . A 2-D Cartesian coordinate system is established to describe horizontal locations of UAVs and GUs, where the central point of the area is located at $[0, 0]^T$. As a result, the horizontal positions of UAV- n and GU- m are denoted as $\mathbf{q}_t^n$ and $\mathbf{w}_t^m$, respectively.

In air-to-ground communications, obstacles such as terrain or buildings often hinder wireless signal propagation, leading to both line-of-sight (LoS) and non line-of-sight (NLoS) links. Therefore, we employ the elevation angle-dependent probabilistic LoS model [11] to calculate channel gain, which considers both LoS and NLoS links. At time slot t , the elevation angle from

GN- m to UAV- n is $\Theta_t^{n,m} = \frac{180^\circ}{\pi} \cdot \arctan\left(\frac{H}{\|\mathbf{w}^m - \mathbf{q}_t^n\|}\right)$. Then, the probability of LoS link is computed by

$$p_t^{\text{LoS},n,m} = \frac{1}{1 + a \exp(-b(\Theta_t^{n,m} - a))}, \quad (1)$$

where a and b are parameters dependent on the environment. The probability of NLoS link is $p_t^{\text{NLoS},n,m} = 1 - p_t^{\text{LoS},n,m}$. For each case, the channel gain is computed as

$$g_t^{\xi,n,m} = \frac{1}{\eta_t^\xi} \left(\frac{4\pi f^c l_t^{n,m}}{c} \right)^{-\alpha}, \quad \xi \in \{\text{LoS}, \text{NLoS}\}, \quad (2)$$

where f^c is the carrier frequency, α is the path loss exponent, η_t^ξ is the excessive path loss coefficients, and c is the light speed. The final channel gain is computed as the expectation of both LoS and NLoS cases: $g_t^{n,m} = p_t^{\text{LoS},n,m} g_t^{\text{LoS},n,m} + p_t^{\text{NLoS},n,m} g_t^{\text{NLoS},n,m}$.

At each time slot t , each GU associates with the UAV offering the best channel quality to establish a communication link. Each UAV can only serve one associated GU per time slot. We define a user scheduling variable $\beta_t^{n,m}$, where $\beta_t^{n,m} = 1$ if GU- m is scheduled by UAV- n at time slot t and $\beta_t^{n,m} = 0$ otherwise. Following references [5], [7], [8], [11], we assume that all UAVs communicate using the same frequency spectrum. To mitigate interference, UAVs employ directional antennas, constraining the transmitted signal within an angle. This results in a coverage range R_{comm} . We define a coverage relation indicator $c_t^{n,m} \in \{0, 1\}$, where $c_t^{n,m} = 1$ if GU- m is within the coverage area of UAV- n , and $c_t^{n,m} = 0$ otherwise. The transmit power of UAVs is denoted as p , and the bandwidth is represented as B .

The signal-to-noise-plus-interference ratio (SINR) from UAV- n to GU- m is calculated as

$$\text{SINR}_t^{n,m} = \frac{c_t^{n,m} \beta_t^{n,m} p g_t^{n,m}}{N_0 B + \sum_{n'} \sum_{m' \neq m} c_t^{n',m} \beta_t^{n',m'} p g_t^{n',m'}}, \quad (3)$$

where N_0 represents the power spectral density (PSD) of additive white Gaussian noise (AWGN). Then, the achievable throughput from UAV- n to GU- m at time slot t is computed as $\delta_t^{n,m} = B \log_2(1 + \text{SINR}_t^{n,m})\tau$. Consequently, the total achievable throughput of GU- m and the entire system over t time slots are given by $\Gamma_t^m = \sum_{i=1}^t \sum_{n=1}^N \delta_i^{n,m}$ and $\Gamma_t = \sum_{i=1}^t \sum_{m=1}^M \sum_{n=1}^N \delta_i^{n,m}$, respectively. Additionally, the average throughput achieved by GU- m over the previous t time slots is $\bar{\Gamma}_t^m = \Gamma_t^m / t$.

B. Problem Formulation

We aim to maximize the system throughput while ensuring fairness among all GUs. Following reference [11], we introduce Jain's fairness index to evaluate fairness among GUs. Specifically, we define a throughput fairness index at each time slot t based on each GU's average throughput over preceding time slots:

$$\eta_t = \frac{\left(\sum_{m=1}^M \bar{\Gamma}_t^m\right)^2}{M \left(\sum_{m=1}^M (\bar{\Gamma}_t^m)^2\right)}. \quad (4)$$

η_t always satisfies $0 \leq \eta_t \leq 1$. A higher fairness index indicates narrower variations in throughput among different GUs. The

maximum value of η_t is reached when all GUs achieve the same average throughput.

We define the fair throughput as the product of the system throughput and fairness index per time slot, expressed as

$$\delta_t^{\text{fair}} = \eta_t \sum_{m=1}^M \delta_t^m. \quad (5)$$

Meanwhile, the total fair throughput during the communication period can be expressed as $\Gamma_T^{\text{fair}} = \sum_t^T \delta_t^{\text{fair}}$.

We represent the variables for UAV trajectories and user scheduling as $\mathbf{q} = \{\mathbf{q}_t^n | n \in \mathcal{N}, t \in \mathcal{T}\}$ and $\beta = \{\beta_t^{n,m} | n \in \mathcal{N}, m \in \mathcal{M}, t \in \mathcal{T}\}$, respectively. These variables are jointly optimized to maximize the total fair throughput during the communication period. The corresponding optimization problem can be formulated as follows:

$$\max_{\mathbf{q}_T, \beta_T} \Gamma_T^{\text{fair}} \quad (6)$$

$$\text{s.t. } \|\mathbf{q}_{t+1}^n - \mathbf{q}_t^n\| = V\tau, \quad n \in \mathcal{N}, t \in \mathcal{T} \quad (6a)$$

$$\sum_{m \in \mathcal{M}} \beta_t^{n,m} \leq 1, \quad n \in \mathcal{N}, t \in \mathcal{T}. \quad (6b)$$

Constraint (6a) enforces constant-speed UAV motion, while constraint (6b) specifies that each UAV can schedule at most one GU at any time slot. This problem is an NP-hard mixed-integer nonlinear programming problem, involving both continuous trajectory variables and discrete user scheduling variables. The coupling among these variables leads to an extensive solution space. Moreover, the non-convex nature of the objective function further adds the complexity.

In multi-UAV assisted communications, gathering global information involves high communication costs, and central controllers may be absent. Therefore, it is essential to employ a decentralized approach for optimization, where each UAV makes individual decisions. Besides, several additional restrictions must be considered:

- 1) Due to their limited sensing and communication ability, UAVs can only receive partial environment information.
- 2) To cope with potential sudden changes in the environment, the decisions must be made in real time.
- 3) Given the substantial population sizes in practical systems, the approach should be scalable.

To tackle these issues, we convert this optimization problem into a Dec-POMDP and propose a MADRL-based solution.

IV. MADRL FORMULATION WITH SYMMETRIES

In this section, we first formulate a Dec-POMDP and briefly introduce the background knowledge related to symmetries. Subsequently, we employ this formulation to establish a mathematical model for global and distributed symmetries in MADRL. Finally, we identify three symmetries present in multi-UAV assisted communications.

A. Entity-Factored Dec-POMDP Formulation

Within the Dec-POMDP formulation, we treat UAVs as individual agents. At each time slot, the environment information is described by state $\mathbf{s}_t \in \mathcal{S}$. Each UAV agent- n receives

an observation $\mathbf{o}_t^n \in \mathcal{O}$, which provides partial information about $\mathbf{s}_t$. Then, it selects an action $a_t^n \in \mathcal{A}$ based on its policy $\hat{\pi}(a_t^n | \mathbf{o}_t^n) : \mathcal{O} \times \mathcal{A} \rightarrow [0, 1]$. All agents' actions form a joint action $\mathbf{a}_t = [a_t^1, \dots, a_t^N] \in \mathcal{A} = \mathcal{A} \times \dots \times \mathcal{A}$. As a result, the environment transitions to the next state $\mathbf{s}_{t+1}$ based on the state transition probability function $P(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$. Meanwhile, all agents receive a reward $r_t = R(\mathbf{s}_t, \mathbf{a}_t)$. The goal of MADRL is to determine the optimal policies for all agents that maximize the expected cumulative reward $\mathbb{E}_{\pi}[\sum_{l=0}^{\infty} \gamma^l r_{t+l}]$, where $\gamma \in [0, 1)$ denotes the discount factor.

To facilitate the subsequent conversion of global symmetries into distributed symmetries, we adopt an entity-factored approach to model Dec-POMDP [31]. Specifically, multi-UAV assisted communication systems comprise two types of entities: UAVs as learning agents and GUs as non-learning entities. Each entity's feature is represented as a vector containing relevant environment information. Specifically, UAV- n 's feature at time slot t contains its current position and whether it schedules any GU at last time slot, i.e.,

$$\mathbf{x}_t^{\text{uav},n} = \left[\mathbf{q}_t^n, \sum_{m=1}^M \beta_{t-1}^{n,m} \right]. \quad (7)$$

Similarly, GU- m 's feature includes its position, whether it is scheduled by any UAV at last time slot, and its average throughput achieved over previous t time slots, i.e.,

$$\mathbf{x}_t^{\text{user},m} = \left[\mathbf{w}_t^m, \sum_{n=1}^N \beta_{t-1}^{n,m}, \bar{\Gamma}_t^m \right]. \quad (8)$$

In the following, we define entity-factored components of the Dec-POMDP using these features.

1) *State*: The state encompasses all environment information relevant to the problem, including features of all entities. Consequently, the state is represented as

$$\mathbf{s}_t = \left[\mathbf{x}_t^{\text{uav},1}, \dots, \mathbf{x}_t^{\text{uav},N}, \mathbf{x}_t^{\text{user},1}, \dots, \mathbf{x}_t^{\text{user},M} \right]. \quad (9)$$

2) *Observation*: Each UAV agent can observe entity features within a certain range through sensing and information exchange via control channels. This range is referred to as the sensing range, denoted by R_{sense} . We define the actual feature of UAV- n ' / GU- m perceived by UAV- n as $\tilde{\mathbf{x}}_t^{n,\text{uav},n'} / \tilde{\mathbf{x}}_t^{n,\text{user},m}$, which equals to $\mathbf{x}_t^{\text{uav},n'} / \mathbf{x}_t^{\text{user},m}$ if UAV- n ' / GU- m is within UAV- n 's sensing range and otherwise is set to an all-zero vector. The observation of UAV- n is represented as

$$\mathbf{o}_t^n = \left[\mathbf{x}_t^{\text{uav},n}, \tilde{\mathbf{x}}_t^{n,\text{uav},1}, \dots, \tilde{\mathbf{x}}_t^{n,\text{uav},N}, \tilde{\mathbf{x}}_t^{n,\text{user},1}, \dots, \tilde{\mathbf{x}}_t^{n,\text{user},M} \right]. \quad (10)$$

3) *Action*: At each time slot, UAVs should choose a flying direction to adjust their positions and schedule a GU to communicate with. Accordingly, the action taken by UAV- n is defined as

$$a_t^n = [\phi_t^n, m_t^n], \quad (11)$$

where $\phi_t^n \in [0, 2\pi)$ denotes UAV- n 's flying direction at time slot t , $m_t^n \in \mathcal{M}^+ = \mathcal{M} \cup \{0\}$ is the ID of GU scheduled by UAV- n , and $m_t^n = 0$ indicates no GU is scheduled. Most MADRL

algorithms are designed specifically for either discrete action spaces or continuous action space. Optimizing discrete policies is typically simpler and exhibits better convergence performance compared to continuous policies. Therefore, when agents involve both continuous and discrete actions, the common practice is to discretize the continuous actions [11], [17], [18], [41]. Without loss of generality, we discretize the continuous direction space $[0, 2\pi)$ into 4 directions, denoted as $\Phi = \{0, \frac{\pi}{2}, \pi, \frac{3\pi}{2}\}$.¹ Consequently, each UAV's action space becomes the Cartesian product of the direction set Φ and the GU set $\mathcal{M}$, i.e.,

$$\mathcal{A} = \Phi \times \mathcal{M}^+ = \Phi \times (\mathcal{M} \cup \{0\}). \quad (12)$$

Each action $a^{m,\phi} \in \mathcal{A}$ is a unique combination of a flying direction ϕ from Φ and a GU ID m from $\mathcal{M}^+$. According to the relationship between actions and entities, we factor the action space into $1 + M$ subspaces:

$$\mathcal{A} = \mathcal{A}^{\text{env}} \cup \{\mathcal{A}^{\text{user},m} | m \in \mathcal{M}\}. \quad (13)$$

Here, $\mathcal{A}^{\text{env}} = \Phi \times \{0\} = \{a^{0,0}, a^{0,\frac{\pi}{2}}, a^{0,\pi}, a^{0,\frac{3\pi}{2}}\}$ and $\mathcal{A}^{\text{user},m} = \Phi \times \{m\} = \{a^{m,0}, a^{m,\frac{\pi}{2}}, a^{m,\pi}, a^{m,\frac{3\pi}{2}}\}$ represent entity-uncorrelated action subspace and GU- m -correlated action subspace, respectively. Actions in $\mathcal{A}^{\text{env}}$ only select flying directions without scheduling any GUs, thereby impacting the entire environment but not specific entities. In contrast, actions in $\mathcal{A}^{\text{user},m}$ correspond to scheduling GU- m and choosing a flying direction from Φ , thus impacting GU- m 's feature.

4) *Reward*: Since our goal is to maximize the total fair throughput during the communication period, we define the reward as the fair throughput at each time slot, expressed as

$$r_t = \eta_t \sum_{m=1}^M \sum_{n=1}^N \delta_t^{n,m} \tau. \quad (14)$$

In MADRL, rewards are utilized to train agents' policies and are not necessary for the agents' decision-making processes. Therefore, even if the rewards rely on global environmental information, they will not impact the agents' decentralized decision-making.

5) *Policy*: The policy of each agent- n is represented as the probability $\hat{\pi}(a_t^n | \mathbf{o}_t^n)$ of executing action a_t^n given the observation $\mathbf{o}_t^n$. We construct a policy vector for each agent- n by concatenating probabilities of all discrete actions:

$$\hat{\pi}(\mathbf{o}_t^n) = \left[\hat{\pi}(a^{0,0} | \mathbf{o}_t^n), \dots, \hat{\pi}(a^{M,\frac{3\pi}{2}} | \mathbf{o}_t^n) \right], \quad (15)$$

where $a^{0,0}, a^{0,\frac{\pi}{2}}, a^{0,\pi}, a^{0,\frac{3\pi}{2}}, \dots, a^{M,0}, a^{M,\frac{\pi}{2}}, a^{M,\pi}, a^{M,\frac{3\pi}{2}}$ are all possible discrete actions in the action space $\mathcal{A}$. According to the factorization of the action space, we similarly partition each individual policy vector into $M + 1$ subpolicy vectors:

$$\hat{\pi}(\mathbf{o}_t^n) = \left[\hat{\pi}_t^{n,\text{env}}, \hat{\pi}_t^{n,1}, \dots, \hat{\pi}_t^{n,M} \right], \quad (16)$$

¹The quantization precision can be adjusted arbitrarily. Theoretically, higher quantization precision leads to better performance of optimal policies; however, it also makes solving for these policies more challenging. As this paper primarily investigates the impact of symmetries, this aspect will not be explored in depth.

where

$$\dot{\pi}_t^{n,env} = \left[\dot{\pi}(a^{0,0}|\mathbf{o}_t^n), \dot{\pi}(a^{0,\frac{\pi}{2}}|\mathbf{o}_t^n), \right. \\ \left. \dot{\pi}(a^{0,\pi}|\mathbf{o}_t^n), \dot{\pi}(a^{0,\frac{3\pi}{2}}|\mathbf{o}_t^n) \right]$$

and

$$\dot{\pi}_t^{n,m} = \left[\dot{\pi}(a^{m,0}|\mathbf{o}_t^n), \dot{\pi}(a^{m,\frac{\pi}{2}}|\mathbf{o}_t^n), \right. \\ \left. \dot{\pi}(a^{m,\pi}|\mathbf{o}_t^n), \dot{\pi}(a^{m,\frac{3\pi}{2}}|\mathbf{o}_t^n) \right]$$

denote subpolicy vectors for entity-uncorrelated actions and GU- m -correlated actions, respectively.

The joint policy $\dot{\pi}(\mathbf{a}_t|\mathbf{s}_t)$ is the probability of all agent jointly executing the action $\mathbf{a}_t = [a_t^1, \dots, a_t^N]$ given the state $\mathbf{s}_t$. It can be calculated as the product of probabilities of all agents' individual actions: $\dot{\pi}(\mathbf{a}_t|\mathbf{s}_t) = \prod_{n=1}^N \dot{\pi}(a_t^n|\mathbf{s}_t) \approx \prod_{n=1}^N \dot{\pi}(a_t^n|\mathbf{o}_t^n)$. For the joint policy, we also construct a vector by concatenating all individual agents' policy vectors:

$$\dot{\pi}(\mathbf{s}_t) = [\dot{\pi}(\mathbf{o}_t^1), \dots, \dot{\pi}(\mathbf{o}_t^N)]. \quad (17)$$

B. Preliminaries of Symmetries

Group theory [42] provides a mathematical framework for describing symmetries. This subsection briefly introduces the core concepts of group theory related to symmetries.

1) *Groups and Transformations*: First, the concepts of the group and group action are introduced.

Definition 1 (Group): A group is an algebraic structure containing a set of elements G and a binary operation $\cdot$ defined between elements, satisfying the following group axioms:

- *Closure*: For any two elements g_1 and g_2 within the set, the operation result $g_1 \cdot g_2$ also belongs to the set.
- *Associativity*: For any three elements g_1, g_2 , and g_3 within the set, it holds that $(g_1 \cdot g_2) \cdot g_3 = g_1 \cdot (g_2 \cdot g_3)$.
- *Identity Element*: There exists an identity element e such that for any element g in the set, $e \cdot g = g \cdot e = g$.
- *Inverse Element*: Every element g in the set has an inverse element g^{-1} , satisfying $g \cdot g^{-1} = g^{-1} \cdot g = e$.

Each element in the group can represent a specific transformation. In symmetry analysis, a group represents the set of all symmetric transformations, including permutations, rotations, reflections, etc. Group action describes how the transformations in a group map elements of a specific set (or space) to other elements within the same set (or space).

Definition 2 (Group Action): The action of a group G on a set (or space) X is a mapping $G \times X \rightarrow X$, typically denoted as $(g, x) \mapsto L_g[x]$, satisfying the following two conditions:

- *Compatibility*: For all $g_1, g_2 \in G$ and all $x \in X$, it holds that $L_{g_1 \cdot g_2}[x] = L_{g_1}[L_{g_2}[x]]$.
- *Identity Transformation*: For all $x \in X$, it holds that $L_e[x] = x$, where e is the identity element of G .

2) *Invariance and Equivariance*: Invariance and equivariance are two concepts closely related to symmetries.

Definition 3 (Invariance): Given a transformation $L_g : \mathcal{X} \rightarrow \mathcal{X}$ within a group G , and a function $f : \mathcal{X} \rightarrow \mathcal{Y}$, if for all $g \in G$ and $x \in \mathcal{X}$, f satisfies the following condition:

$$f(x) = f(L_g[x]) \quad (18)$$

then f is said to be *invariant* or *symmetric* with respect to $\{L_g\}_{g \in G}$. The set $\{L_g\}_{g \in G}$ constitutes a group of **symmetric transformations**, and G is called the **symmetry group** of f .

Definition 4 (Equivariance): Given a transformation $L_g : \mathcal{X} \rightarrow \mathcal{X}$ in a group G and a function $f : \mathcal{X} \rightarrow \mathcal{Y}$, if there exists another transformation $K_g : \mathcal{Y} \rightarrow \mathcal{Y}$ such that for all $g \in G$ and $x \in \mathcal{X}$, the following condition is satisfied:

$$K_g[f(x)] = f(L_g[x]) \quad (19)$$

then f is said to be *equivariant* with respect to L_g .

Essentially, symmetry refers to the property of a function where the output remains unchanged after the input undergoes a specific transformation, making invariance a key aspect of symmetry analysis. Equivariance ensures that transformed data maintain the same form across different spaces, which is equally significant in symmetry analysis.

C. Mathematical Model for Symmetries in Dec-POMDP

In this subsection, we first define global symmetries in the joint state-action and then distribute them into symmetries at the agent level.

1) *Global Symmetries in Dec-POMDP*: We define global symmetries using an approach similar to that employed for constructing symmetries in a single-agent MDP [23].

Definition 5: (Symmetric Dec-POMDP): A symmetric Dec-POMDP is a Dec-POMDP in which the following conditions are satisfied for at least one group G of transformations $L_g : \mathcal{S} \rightarrow \mathcal{S}$ and $K_g : \mathcal{A} \rightarrow \mathcal{A}$:

$$P(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t) = P(L_g[\mathbf{s}_{t+1}]|L_g[\mathbf{s}_t], K_g[\mathbf{a}_t]),$$

$$R(\mathbf{s}_t, \mathbf{a}_t) = R(L_g[\mathbf{s}_t], K_g[\mathbf{a}_t]), \forall g \in G, \forall \mathbf{s}_t \in \mathcal{S}, \forall \mathbf{a}_t \in \mathcal{A}. \quad (20)$$

Then, we say $\{(L_g, K_g)\}_{g \in G}$ is a group of **symmetry transformations**, and G is a **symmetry group** of this Dec-POMDP.

In a symmetric Dec-POMDP, applying symmetry transformations to a state-action pair results in a set of equivalent state-action pairs sharing the same reward and state transition probability. Next, we will demonstrate that the optimal policy for a symmetric Dec-POMDP remains invariant to these equivalent pairs.

Theorem 1 (Invariant Joint Policy): For a symmetric Dec-POMDP with respect to $\{(L_g, K_g)\}_{g \in G}$, the optimal joint policy satisfies the following invariance constraint:

$$\dot{\pi}^*(\mathbf{a}_t|\mathbf{s}_t) = \dot{\pi}^*(K_g[\mathbf{a}_t]|L_g[\mathbf{s}_t]), \forall g \in G, \forall \mathbf{s}_t \in \mathcal{S}, \forall \mathbf{a}_t \in \mathcal{A}. \quad (21)$$

Proof: Please refer to the proof of Theorem 2 in [23]. ■

Theorem 1 reveals that the optimal joint policy for a symmetric Dec-POMDP is invariant to the state-action transformations $\{(L_g, K_g)\}_{g \in G}$. Given $\{(L_g, K_g)\}_{g \in G}$, we can incorporate the invariance constraint (21) into the joint policy, thereby reducing

the solution space. However, since our task is to determine each agent's individual policy, we have to transform global symmetries into distributed symmetries.

2) *Distributed Symmetries in Dec-POMDP*: Next, we convert the global symmetries into a set of distributed symmetries at the agent level and establish a corresponding equivariance constraint for individual policies.

In Dec-POMDPs, since the observation of each agent represents part of the state, the transformation of $\mathbf{s}_t$ with L_g results in corresponding transformations in observations:

$$L_g[\mathbf{s}_t] \Rightarrow U_g[\mathbf{o}_t^n], \forall n \in \mathcal{N}, \quad (22)$$

where U_g represents the observation transformation. Consequently, transforming $\mathbf{s}_t$ leads to the following transformation of the joint policy vector:

$$\dot{\pi}(L_g[\mathbf{s}_t]) = [\dot{\pi}(U_g[\mathbf{o}_t^1]), \dots, \dot{\pi}(U_g[\mathbf{o}_t^N])]. \quad (23)$$

Regarding the joint action, acting on $\mathbf{a}_t$ with transformation K_g is equivalent to acting on individual actions with transformation k_g :

$$K_g[\mathbf{a}_t] = [k_g[a_t^1], \dots, k_g[a_t^N]]. \quad (24)$$

For discrete actions, an action transformation k_g maps a discrete action to another discrete action invertibly, leading to the permutation of elements within each agent's policy vector:

$$\mathbf{k}_g \dot{\pi}(\mathbf{o}_t^n) = [\dot{\pi}(k_g[a^{0,0} | \mathbf{o}_t^n]), \dots, \dot{\pi}(k_g[a^{M, \frac{3\pi}{2}} | \mathbf{o}_t^n])], \quad (25)$$

where $\mathbf{k}_g$ represents the individual policy permutation matrix resulting from the action transformation k_g . Correspondingly, the joint policy vector is also permuted as follows:

$$\mathbf{K}_g \dot{\pi}(\mathbf{s}_t) = [\mathbf{k}_g \dot{\pi}(\mathbf{o}_t^1), \dots, \mathbf{k}_g \dot{\pi}(\mathbf{o}_t^N)], \quad (26)$$

where $\mathbf{K}_g$ denotes the joint policy permutation matrix caused by permutations of individual policy vectors.

The above derivations lead to an equivariance property of individual policies, as stated below.

Proposition 1 (Equivariant Individual Policies): For a symmetric Dec-POMDP, the optimal individual policies satisfy the following equivariance constraint:

$$\mathbf{k}_{g^{-1}} \dot{\pi}^*(\mathbf{o}_t^n) = \dot{\pi}^*(U_g[\mathbf{o}_t^n]), \forall g \in G, \forall n \in \mathcal{N}, \forall \mathbf{o}_t^n \in \mathcal{O}, \quad (27)$$

where $\mathbf{k}_{g^{-1}}$ represents the inverse matrix of $\mathbf{k}_g$.

Proof: Since we use the joint policy vector to represent the joint policy, the invariance constraint (21) can be written as

$$\dot{\pi}^*(\mathbf{s}_t) = \mathbf{K}_g \dot{\pi}^*(L_g[\mathbf{s}_t]). \quad (28)$$

Based on equations (23) and (26), this expression can be further represented as

$$\begin{aligned} & [\dot{\pi}^*(\mathbf{o}_t^1), \dots, \dot{\pi}^*(\mathbf{o}_t^N)] \\ &= [\mathbf{k}_g \dot{\pi}^*(U_g[\mathbf{o}_t^1]), \dots, \mathbf{k}_g \dot{\pi}^*(U_g[\mathbf{o}_t^N])]. \end{aligned} \quad (29)$$

Therefore, we have $\dot{\pi}^*(\mathbf{o}_t^n) = \mathbf{k}_g \dot{\pi}^*(U_g[\mathbf{o}_t^n])$, which can be further transformed into constraint (27). ■

Proposition 1 indicates that the optimal individual policies for a symmetric Dec-POMDP are equivariant to the observation-policy transformation $(U_g, \mathbf{k}_g)$, i.e., the transformation of the

observation with U_g results in a corresponding transformation of individual policy vector with $\mathbf{k}_g$. Given $\{(U_g, \mathbf{k}_g)\}_{g \in G}$, we can incorporate the equivariance constraint (27) into individual policies, reducing the solution space.

D. Symmetries in Multi-UAV Assisted Communications

Based on the Dec-POMDP model constructed in Section IV-A, we identify three symmetries present in multi-UAV assisted communications: permutation symmetry, rotational symmetry, and reflection symmetry.

1) *Permutation Symmetry*: In multi-UAV assisted communications, UAVs and GUs do not have a natural order. Therefore, state-action pairs with identical entity features but different entity orders are equivalent. According to this prior knowledge, we define the permutation symmetry.

All permutations of N UAVs and M GUs form a symmetry group G^{pt} . Given any permutation-pair $(\omega_g^{\text{uav}}, \omega_g^{\text{user}}) \in G^{\text{pt}} : \mathcal{N} \times \mathcal{M} \rightarrow \mathcal{N} \times \mathcal{M}$, the orders of UAV- n and GU- m become $n'' = \omega_g^{\text{uav}}[n]$ and $\omega_g^{\text{user}}[m]$, respectively. Additionally, entity features within the observation vector is also permuted, resulting in a new observation vector $\bar{\mathbf{o}}_t^{n''}$. We denote the inverse operation of ω_g^{uav} and ω_g^{user} as $\omega_{g^{-1}}^{\text{uav}}$ and $\omega_{g^{-1}}^{\text{user}}$, respectively. Then, the observation transformation can be represented as

$$\begin{aligned} \bar{\mathbf{o}}_t^{n''} = U_g^{\text{pt}}[\mathbf{o}_t^n] = & \left[\mathbf{x}_{t^{\text{uav},n}}^{\text{uav}}, \tilde{\mathbf{x}}_t^{n,\text{uav},\omega_{g^{-1}}^{\text{uav}}[1]}, \dots, \tilde{\mathbf{x}}_t^{n,\text{uav},\omega_{g^{-1}}^{\text{uav}}[N]}, \right. \\ & \left. \tilde{\mathbf{x}}_t^{n,\text{user},\omega_{g^{-1}}^{\text{user}}[1]}, \dots, \tilde{\mathbf{x}}_t^{n,\text{uav},\omega_{g^{-1}}^{\text{user}}[M]} \right]. \end{aligned} \quad (30)$$

It should be noted that $\bar{\mathbf{o}}_t^{n''}$ is obtained by reordering entity features in $\mathbf{o}_t^n$, without changing individual entity features.

According to (16), each agent's policy vector consists of an entity-uncorrelated subpolicy vector and M GU-correlated subpolicy vectors. On the one hand, due to the correspondence between each GU- m -correlated subpolicy vector and GU- m , permuting GUs' order leads to the same permutation among different subpolicy vectors related to these GUs. On the other hand, the choice of flying direction is independent of the UAV and GU orders. Therefore, permuting UAVs or GUs' features does not change the action probabilities related to different flying directions within each subpolicy. Consequently, the corresponding individual policy permutation matrix can be represented as

$$\mathbf{k}_g^{\text{pt}} \dot{\pi}(\mathbf{o}_t^n) = [\dot{\pi}_t^{n,\text{env}}, \dot{\pi}_t^{n,\omega_g^{\text{user}}[1]}, \dots, \dot{\pi}_t^{n,\omega_g^{\text{user}}[M]}]. \quad (31)$$

Based on the above formulation, the equivariance constraint (27) can be specified as the following permutation equivariance constraint:

$$\mathbf{k}_{g^{-1}}^{\text{pt}} \dot{\pi}(\mathbf{o}_t^n) = \dot{\pi}(U_g^{\text{pt}}[\mathbf{o}_t^n]), \forall g \in G^{\text{pt}}, \forall n \in \mathcal{N}, \forall \mathbf{o}_t^n \in \mathcal{O}. \quad (32)$$

2) *Rotational and Reflection Symmetries*: In multi-UAV assisted communications, rotational and reflection symmetries demonstrate the system's invariant properties under specific spatial transformations. As described in Section III-A, in the probabilistic LoS model, the channel gain between UAVs and

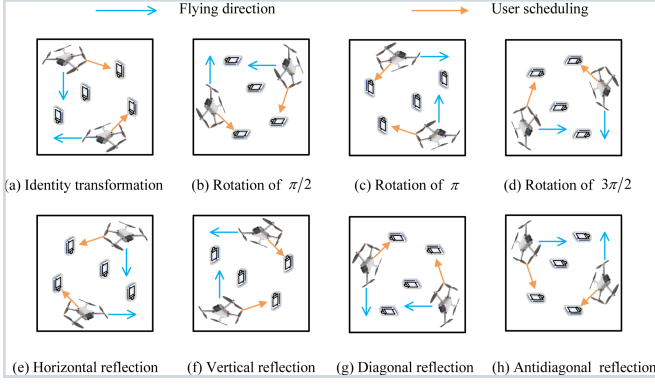


Fig. 3. Illustrations of rotational and reflection symmetries, where UAVs and GUs are projected onto the same plane.

GNs is determined solely by their relative positions, not by their absolute positions.² Consequently, as long as the relative positions between UAVs and GNs remains unchanged, UAVs' policies should also remain unchanged.

The space encompassing all UAVs and GUs forms a cuboid, where UAVs and GUs operate on planes at different altitudes, restricted to horizontal movement only. In this case, positional changes in three-dimensional space are reduced to movements on a two-dimensional plane. Specifically, when the planes containing UAVs and GUs rotate around the central point by the same angle or reflect along the same symmetry axis, the horizontal positions of these entities change accordingly. However, their relative positions remain constant, preserving the quality of the ground-to-air communication channel. As a result, the scheduling relationship between UAVs and GUs, as well as the UAVs' flying direction relative to their service area should also remain unchanged. This indicates that the state-action pairs, after symmetric rotations or reflections, are equivalent in terms of the system's reward and state transition probabilities. Leveraging this prior knowledge, we can construct the corresponding symmetry group.

To clarify, we project the positions of UAVs and GUs onto a single plane, representing their movements as horizontal shifts on this plane. As illustrated in Fig. 3, within this plane, all symmetric rotations and reflections for a square area form a symmetry group. This group comprises an identity transformation g_e , rotations at three angles: $g_{\pi/2}$, g_{π} , and $g_{3\pi/2}$, as well as reflections across four symmetry axis: horizontal g_H , vertical g_V , diagonal g_{D1} , and antidiagonal g_{D2} . We represent this group as $G^{\text{rt}} = \{g_e, g_{\pi/2}, g_{\pi}, g_{3\pi/2}, g_H, g_V, g_{D1}, g_{D2}\}$.

²This represents an idealized channel model: in real communication systems, the channel quality between communication nodes depends not only on their relative positions but also on the surrounding environment. In such scenarios, it is necessary to incorporate spatial and geographical information when constructing state-action space and rediscover symmetries. However, this paper focuses on exploring symmetries within the existing state-action space to enhance learning algorithms. The construction of a state-action space that includes complex geographical information and the management of non-ideal symmetries will be addressed in future work.

A rotation by an angle ψ acting on a point $\mathbf{z} = [z_1, z_2]^T$ can be achieved using a 2×2 matrix:

$$R_{g_\psi}[\mathbf{z}] = \begin{bmatrix} \cos \psi & -\sin \psi \\ \sin \psi & \cos \psi \end{bmatrix} \mathbf{z}. \quad (33)$$

Similarly, the reflections can be achieved using 2×2 matrices as follows:

$$R_{g_H}[\mathbf{z}] = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \mathbf{z}, \quad R_{g_V}[\mathbf{z}] = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \mathbf{z},$$

$$R_{g_{D1}}[\mathbf{z}] = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \mathbf{z}, \quad R_{g_{D2}}[\mathbf{z}] = \begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix} \mathbf{z}. \quad (34)$$

Rotating or reflecting coordinates of all entities with $g \in G^{\text{rt}}$ results in the following transformations for each UAV and GU's features:

$$l_g^{\text{uav}}[\mathbf{x}_t^{\text{uav},n}] = \left[R_g[\mathbf{q}_t^n], \sum_{m=1}^M \beta_{t-1}^{n,m} \right], \quad \forall n \in \mathcal{N}$$

$$l_g^{\text{user}}[\mathbf{x}_t^{\text{user},m}] = \left[R_g[\mathbf{w}_t^m], \sum_{n=1}^N \beta_{t-1}^{n,m}, \bar{\Gamma}_t^m \right], \quad \forall m \in \mathcal{M}. \quad (35)$$

Consequently, the state transformation can be represented as

$$L_g^{\text{rt}}[\mathbf{s}_t] = \left[l_g^{\text{uav}}[\mathbf{x}_t^{\text{uav},1}], \dots, l_g^{\text{uav}}[\mathbf{x}_t^{\text{uav},N}], \right. \\ \left. l_g^{\text{user}}[\mathbf{x}_t^{\text{user},1}], \dots, l_g^{\text{user}}[\mathbf{x}_t^{\text{user},M}] \right]. \quad (36)$$

Similarly, the observation transformation can be represented as

$$U_g^{\text{rt}}[\mathbf{o}_t^n] = \left[l_g^{\text{uav}}[\mathbf{x}_t^{\text{uav},n}], l_g^{\text{uav}}[\tilde{\mathbf{x}}_t^{n,\text{uav},1}], \dots, l_g^{\text{uav}}[\tilde{\mathbf{x}}_t^{n,\text{uav},N}], \right. \\ \left. l_g^{\text{user}}[\tilde{\mathbf{x}}_t^{n,\text{user},1}], \dots, l_g^{\text{user}}[\tilde{\mathbf{x}}_t^{n,\text{user},M}] \right]. \quad (37)$$

For the individual action, a rotation of ψ transforms the flying direction ϕ corresponding to each discrete action $a^{m,\phi}$ to $(\phi + \psi) \bmod 2\pi$, where $\bmod$ denotes the modulus operation. Consequently, their corresponding individual action transformation can be represented as

$$k_{g_\psi}^{\text{rt}}[a^{m,\phi}] = a^{m,(\phi+\psi) \bmod 2\pi}, \quad \forall m \in \mathcal{M}^+, \quad \forall \phi \in \Phi. \quad (38)$$

The horizontal reflection, vertical reflection, and two diagonal reflection transform the flying direction ϕ to $(2\pi - \phi) \bmod 2\pi$, $(\pi - \phi) \bmod 2\pi$, $(\frac{\pi}{2} - \phi) \bmod 2\pi$, and $(\frac{3\pi}{2} - \phi) \bmod 2\pi$, respectively. As a result, their corresponding individual action transformation can be represented as

$$\begin{cases} k_{g_H}^{\text{rt}}[a^{m,\phi}] = a^{m,(2\pi-\phi) \bmod 2\pi} \\ k_{g_V}^{\text{rt}}[a^{m,\phi}] = a^{m,(\pi-\phi) \bmod 2\pi} \\ k_{g_{D1}}^{\text{rt}}[a^{m,\phi}] = a^{m,(\frac{\pi}{2}-\phi) \bmod 2\pi} \\ k_{g_{D2}}^{\text{rt}}[a^{m,\phi}] = a^{m,(\frac{3\pi}{2}-\phi) \bmod 2\pi} \end{cases} \quad \forall m \in \mathcal{M}^+, \quad \forall \phi \in \Phi \quad (39)$$

In the next section, we will develop a MADRL approach based on these symmetries.

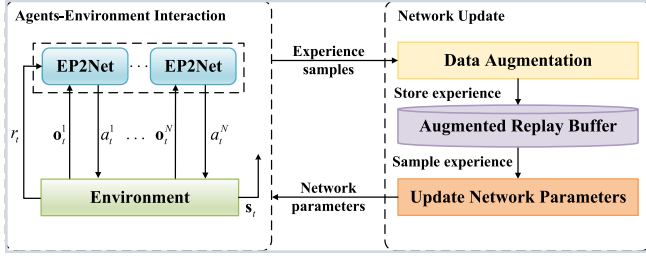


Fig. 4. Overview of the symmetry-augmented MADRL approach. Key distinctions between the proposed and standard MADRL approaches include: (a) The introduction of EP2Nets as policy networks for agents. (b) The incorporation of data augmentation into the learning process.

V. SYMMETRY-AUGMENTED MADRL APPROACH

The typical learning process of MADRL involves two main steps: (1) The collection of experience samples through interactions between agents and the environment. (2) The iterative update of network parameters using these experience samples [36]. As illustrated in Fig. 4, the proposed symmetry-augmented MADRL approach differs from the standard MADRL approaches in two aspects: (1) The introduction of a family of neural networks for learning individual policies, namely entity permutation equivariant policy networks (EP2Nets). (2) The incorporation of data augmentation to expand experience samples. Both aspects leverage symmetries in multi-UAV assisted communications.

In this section, we first outline the structure of EP2Net and its two implementation methods. Subsequently, we discuss the data augmentation method that leverages rotational and reflection symmetries. Finally, we integrate these symmetry methods with the QMIX algorithm to introduce the SymmQMIX algorithm.

A. Entity Permutation Equivariant Policy Networks

An entity permutation equivariant network (EP2Net) estimates the optimal individual policy for a UAV agent- n . It receives the agent's observation vector as input and generates all action probabilities. Based on these probabilities, the final action can be selected stochastically. Compared to traditional policy networks, EP2Nets leverage permutation symmetry to reduce the state-action space, as elaborated below. For more details about permutation symmetry, see Section IV-D1.

1) *The Architecture of EP2Nets*: To ensure that the policy network for each agent remains unchanged when the order of agents changes, an EP2Net is shared across all UAV agents. Then, we only need to guarantee that permuting entity features in the observation vector results in a corresponding permutation in the policy vector. This is achieved by establishing correspondence between entity features and actions.

Fig. 5 illustrates the architecture of an EP2Net, which consists of an input module, a middle module, and an output module. We denote the functions of these layers as $f_{in}(\cdot)$, $f_{mid}(\cdot)$, and $f_{out}(\cdot)$, respectively. Moreover, the function of the whole EP2Net is represented as $\hat{\pi}(\mathbf{o}_t^n) = f_{ep}(\mathbf{o}_t^n)$.

The input module aggregates information from all perceivable entity features in the observation vector to generate an

embedding feature. As shown in Fig. 5(a), EP2Net first generate an individual embedding for each entity feature using three distinct functions. Specifically, $f_{in}^{own}(\cdot)$ is utilized to produce the embedding for the agent's own feature, while $f_{in}^{uav}(\cdot)$ and $f_{in}^{user}(\cdot)$ are shared across different UAVs and GUs to generate their respective embeddings. This process can be expressed as

$$\begin{aligned} \mathbf{z}_t^{n,own} &= f_{in}^{own}(\mathbf{x}_t^{uav,n}), \\ \mathbf{z}_t^{n,uav,n'} &= f_{in}^{uav}(\tilde{\mathbf{x}}_t^{n,uav,n'}), \forall n' \in \mathcal{N} \setminus n, \\ \mathbf{z}_t^{n,user,m} &= f_{in}^{user}(\tilde{\mathbf{x}}_t^{n,user,m}), \forall m \in \mathcal{M}. \end{aligned} \quad (40)$$

Then, all UAVs/GUs' individual embeddings are aggregated using a summation operation and further processed using $g_{in}^{uav}(\cdot)$ / $g_{in}^{user}(\cdot)$ to generate a group embedding:

$$\begin{aligned} \mathbf{z}_t^{n,uav} &= g_{in}^{uav} \left(\sum_{n' \neq n} \mathbf{z}_t^{n,uav,n'} \right), \\ \mathbf{z}_t^{n,user} &= g_{in}^{user} \left(\sum_m \mathbf{z}_t^{n,user,m} \right). \end{aligned} \quad (41)$$

Finally, $g_{in}^{emb}(\cdot)$ takes the agent's own embedding, UAVs' group embedding, and GUs' group embedding as input, and produces the final embedding feature:

$$\mathbf{z}_t^n = g_{in}^{emb}(\mathbf{z}_t^{n,own}, \mathbf{z}_t^{n,uav}, \mathbf{z}_t^{n,user}). \quad (42)$$

The middle module further extracts information from the embedding feature. Following a minimal modification principle, we impose no restrictions on its structure. As in most policy networks, the middle layer produces a hidden state by taking the embedding feature as input:

$$\mathbf{h}_t^n = f_{mid}(\mathbf{z}_t^n). \quad (43)$$

The output module infers the action probabilities by integrating information from the input and middle modules. As depicted in Fig. 5(b), the output module comprises two different functions, denoted as $f_{out}^{env}(\cdot)$ and $f_{out}^{user}(\cdot)$, respectively. We utilize $f_{out}^{env}(\cdot)$ to generate entity-uncorrelated subpolicy vectors by taking the hidden state as input:

$$\hat{\pi}_t^{n,env} = [\hat{\pi}(a^{0,0}|\mathbf{o}_t^n), \dots, \hat{\pi}(a^{0,\frac{3\pi}{2}}|\mathbf{o}_t^n)] = f_{out}^{env}(\mathbf{h}_t^n). \quad (44)$$

Meanwhile, $f_{out}^{user}(\cdot)$ is employed to produce M GU-correlated subpolicy vectors separately. To establish the correspondence between each entity and actions related to it, we introduce GU- m 's perceivable feature as an additional input of $f_{out}^{user}(\cdot)$ when generating GU- m -correlated subpolicy vector. Consequently, each $\hat{\pi}_t^{n,m}$ can be calculated as

$$\begin{aligned} \hat{\pi}_t^{n,m} &= [\hat{\pi}(a^{m,0}|\mathbf{o}_t^n), \dots, \hat{\pi}(a^{m,\frac{3\pi}{2}}|\mathbf{o}_t^n)] \\ &= f_{out}^{user}(\mathbf{h}_t^n, \tilde{\mathbf{x}}_t^{n,user,m}), m \in \mathcal{M}. \end{aligned} \quad (45)$$

These subpolicy vectors are concatenated to form the policy vector $\hat{\pi}(\mathbf{o}_t^n)$ of agent- n :

$$\hat{\pi}(\mathbf{o}_t^n) = [\hat{\pi}_t^{n,env}, \hat{\pi}_t^{n,1}, \dots, \hat{\pi}_t^{n,M}]$$

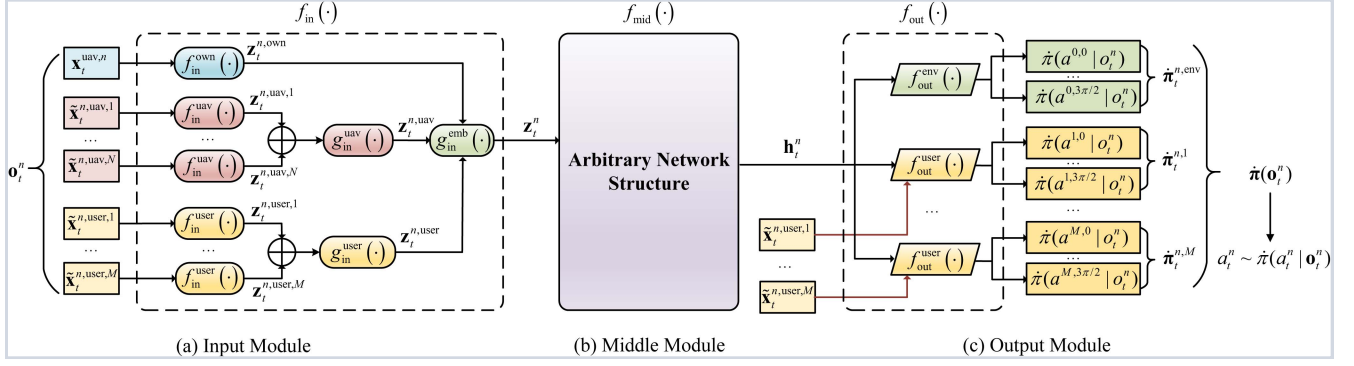


Fig. 5. The overall architecture of an EP2Net. (a) The input module generates embeddings for entities individually and then combines them into a final embedding. (b) The structure of the middle module remains unconstrained. (c) The output module generates each subpolicy vector separately.

$$\begin{aligned}
 &= \left[f_{\text{out}}^{\text{env}}(\mathbf{h}_t^n), f_{\text{out}}^{\text{user}}(\mathbf{h}_t^n, \tilde{\mathbf{x}}_t^{n,\text{user},1}), \dots, f_{\text{out}}^{\text{user}}(\mathbf{h}_t^n, \tilde{\mathbf{x}}_t^{n,\text{user},M}) \right] \\
 &= f_{\text{out}}(\mathbf{h}_t^n, [\tilde{\mathbf{x}}_t^{n,\text{user},1}, \dots, \tilde{\mathbf{x}}_t^{n,\text{user},M}]) = f_{\text{ep}}(\mathbf{o}_t^n). \quad (46)
 \end{aligned}$$

2) *Properties of EP2Nets*: In the following, we will demonstrate that EP2Net satisfies the permutation equivariance property. Before that, we first present the permutation invariance property of the input module.

Proposition 2 (Permutation Invariance): Given the permutation group G^{pt} and the observation transformation U_g^{pt} , the input module satisfies the permutation invariance constraint:

$$\mathbf{z}_t^n = f_{\text{in}}(U_g^{\text{pt}}[\mathbf{o}_t^n]) = f_{\text{in}}(\mathbf{o}_t^n), \quad \forall g \in G^{\text{pt}}. \quad (48)$$

Proof: According to the definition of U_g^{pt} , $U_g^{\text{pt}}[\mathbf{o}_t^n]$ simply reorders $\{\tilde{\mathbf{x}}_t^{n,\text{uav},n'}\}_{n' \neq n}$ and $\{\tilde{\mathbf{x}}_t^{n,\text{user},m}\}_m$. Since the input module processes each UAV/GU feature separately using a shared function and then merging them through a summation operation, the group embeddings $\mathbf{z}_t^{n,\text{uav}}$ and $\mathbf{z}_t^{n,\text{user}}$ remain unchanged regardless of UAVs'/GUs' arrangement. As a result, the final embedding feature $\mathbf{z}_t^{n,\text{user}}$ is invariant to U_g^{pt} . ■

Corollary 1: The cascade of the permutation-invariant input module with the middle module still maintains permutation invariance with respect to the observation vector:

$$\mathbf{h}_t^n = f_{\text{mid}}(f_{\text{in}}(U_g^{\text{pt}}[\mathbf{o}_t^n])) = f_{\text{mid}}(f_{\text{in}}(\mathbf{o}_t^n)), \quad \forall g \in G^{\text{pt}}. \quad (49)$$

Proposition 3 (Permutation Equivariance): Given the permutation group G^{pt} and the observation-policy transformation pair $(U_g^{\text{pt}}, \mathbf{k}_g^{\text{pt}})$, an EP2Net satisfies the permutation equivariance

constraint:

$$\mathbf{k}_{g-1}^{\text{pt}} f_{\text{ep}}(\mathbf{o}_t^n) = f_{\text{ep}}(U_g^{\text{pt}}[\mathbf{o}_t^n]), \quad \forall g \in G^{\text{pt}}. \quad (50)$$

Proof: According to Proposition 2 and Corollary 1, the hidden state is invariant to U_g^{pt} . By substituting the reordered GUs' perceivable features and $\mathbf{h}_t^n$ into (46), we can obtain the final expression for $f_{\text{ep}}(U_g^{\text{pt}}[\mathbf{o}_t^n])$, as presented in (47) shown at the bottom of this page. The final equality of (47) is derived based on the definition of $\mathbf{k}_{g-1}^{\text{pt}}$, thus establishing the permutation equivariance. ■

Proposition 3 reveals that EP2Nets are equivariant to the permutation of entity orders. In other words, when the entity order within the observation vector is permuted according to U_g^{pt} , the action probabilities generated by EP2Net will undergo a corresponding permutation according to $\mathbf{k}_g^{\text{pt}}$. There are a total of $N! \times M!$ permutations of all UAVs and GUs. Since EP2Nets are equivariant to these permutations, experience gained from one permutation can be utilized to improve the policies for all permutations. Consequently, the state-action space is reduced by a factor of $\frac{1}{N! \times M!}$.

Besides the reduced state-action space, EP2Net has the following advantages:

- 1) *Ability to handle dynamic population sizes*: EP2Nets perform computations in an entity-wise manner, and the function is shared across homogeneous entities. Consequently, EP2Net can handle an arbitrary number of UAV/GU features.
- 2) *Algorithm-agnostic*: In EP2Nets, we only design input and output modules, keeping the backbone middle module unchanged. As a result, EP2Nets can be easily plugged into most MADRL algorithms.

$$\begin{aligned}
 f_{\text{ep}}(U_g^{\text{pt}}[\mathbf{o}_t^n]) &= f_{\text{out}}\left(\mathbf{h}_t^n, \left[\tilde{\mathbf{x}}_t^{n,\text{user},\omega_{g-1}^{\text{user}}[1]}, \dots, \tilde{\mathbf{x}}_t^{n,\text{uav},\omega_{g-1}^{\text{user}}[M]}\right]\right) \\
 &= \left[f_{\text{out}}^{\text{env}}(\mathbf{h}_t^n), f_{\text{out}}^{\text{user}}\left(\mathbf{h}_t^n, \tilde{\mathbf{x}}_t^{n,\text{user},\omega_{g-1}^{\text{user}}[1]}\right), \dots, f_{\text{out}}^{\text{user}}\left(\mathbf{h}_t^n, \tilde{\mathbf{x}}_t^{n,\text{uav},\omega_{g-1}^{\text{user}}[M]}\right) \right] \\
 &= \left[\tilde{\pi}_t^{n,\text{env}}, \tilde{\pi}_t^{n,\omega_{g-1}^{\text{user}}[1]}, \dots, \tilde{\pi}_t^{n,\omega_{g-1}^{\text{user}}[M]} \right] = \mathbf{k}_{g-1}^{\text{pt}} f_{\text{ep}}(\mathbf{o}_t^n) \quad (47)
 \end{aligned}$$

TABLE I
 NETWORK CONFIGURATIONS OF FEP2NET AND HEP2NET

Function	FEP2Net	HEP2Net
$f_{in}^{own}(\cdot)$	A linear layer	
$f_{in}^{uav}(\cdot)$	A linear layer	A hypernetwork-based linear layer
$f_{in}^{user}(\cdot)$	A linear layer	A hypernetwork-based linear layer
$g_{in}^{uav}(\cdot)$	No processing	
$g_{in}^{user}(\cdot)$	No processing	
$g_{in}^{emb}(\cdot)$	Summation operation + Nonlinear activation function	
$f_{out}^{env}(\cdot)$	A linear layer	
$f_{out}^{user}(\cdot)$	A linear layer	A hypernetwork-based linear layer

3) *Flexible network design*: We do not specify the implementations of $f_{in}^{own}(\cdot)$, $f_{in}^{uav}(\cdot)$, $f_{in}^{user}(\cdot)$, $g_{in}^{uav}(\cdot)$, $g_{in}^{user}(\cdot)$, $g_{in}^{emb}(\cdot)$, $f_{out}^{env}(\cdot)$, and $f_{out}^{user}(\cdot)$ in EP2Nets. This grants the flexibility in designing these functions to create different EP2Nets.

B. Efficient Implementations of EP2Nets

This subsection presents two implementations of EP2Net. First, we introduce the most basic version: the fully-connected EP2Net (FEP2Net), which is primarily implemented using linear layers. Next, by introducing hypernetworks into the FEP2Net, we propose the hypernetwork-based EP2Net (HEP2Net), which offers stronger model representational capabilities. Finally, the computational complexity of FEP2Net and HEP2Net is compared. Table I displays the network configurations of each function in FEP2Net and HEP2Net.

1) *Fully-Connected EP2Net*: FEP2Net represents the simplest implementation of EP2Nets. Specifically, we set $f_{in}^{own}(\cdot)$, $f_{in}^{uav}(\cdot)$, and $f_{in}^{user}(\cdot)$ as linear layers for calculating embeddings, represented as $LN_{in}^{own}(\cdot)$, $LN_{in}^{uav}(\cdot)$, and $LN_{in}^{user}(\cdot)$, respectively. Each linear layer produces the output embedding by applying a weight matrix multiplication and adding a bias vector. For $g_{in}^{uav}(\cdot)$ and $g_{in}^{user}(\cdot)$, we set them as identity functions, implying no processing. Regarding $g_{in}^{emb}(\cdot)$, we aggregate the input embeddings and then pass them through a non-linear activation function such as ReLU, denoted as $\sigma_{in}(\cdot)$. As a result, the expression for the input module can be written as

$$\begin{aligned} \mathbf{z}_t^n = f_{in}(\mathbf{o}_t^n) = & \sigma_{in} \left(\mathbf{LN}_{in}^{own}(\mathbf{x}_t^{uav,n}; \theta_{in}^{own}) \right. \\ & \left. + \sum_{n' \neq n} \mathbf{LN}_{in}^{uav}(\tilde{\mathbf{x}}_t^{n,uav,n'}; \theta_{in}^{uav}) + \sum_m \mathbf{LN}_{in}^{user}(\tilde{\mathbf{x}}_t^{n,user,m}; \theta_{in}^{user}) \right), \end{aligned} \quad (51)$$

where θ_{in}^{own} , θ_{in}^{uav} , and θ_{in}^{user} denote the learnable parameters of linear layers including weights and biases. In the output module, we set both $f_{out}^{user}(\cdot)$ and $f_{out}^{env}(\cdot)$ as linear layers, denoted as $LN_{out}^{user}(\cdot)$ and $LN_{out}^{env}(\cdot)$. Consequently, the expression for the output module can be written as

$$\begin{aligned} \dot{\pi}(\mathbf{o}_t^n) = & \left[\mathbf{LN}_{out}^{env}(\mathbf{h}_t^n; \theta_{out}^{env}), \mathbf{LN}_{out}^{user}(\mathbf{h}_t^n, \tilde{\mathbf{x}}_t^{n,user,1}; \theta_{out}^{user}) \right. \\ & \left. \dots, \mathbf{LN}_{out}^{user}(\mathbf{h}_t^n, \tilde{\mathbf{x}}_t^{n,user,M}; \theta_{out}^{user}) \right], \end{aligned} \quad (52)$$

where θ_{out}^{env} and θ_{out}^{user} represent network parameters.

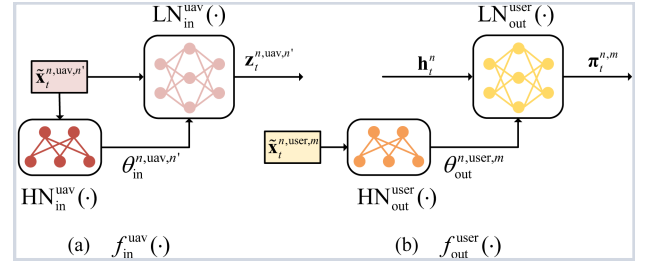


Fig. 6. Illustration of the hypernetwork-based network implementation. (a) In the implementation of $f_{in}^{uav}(\cdot)$, the hypernetwork $HN_{in}^{uav}(\cdot)$ takes each UAV feature as input and generates network parameters for the linear layer $LN_{in}^{uav}(\cdot)$. (b) In the implementation of $f_{out}^{user}(\cdot)$, the hypernetwork $HN_{out}^{user}(\cdot)$ takes each GU feature as input and generates network parameters for $LN_{out}^{user}(\cdot)$.

In this setup, the input module essentially functions as an FC layer that shares weights and biases among different UAVs/GUs. Meanwhile, the output module acts as a linear layer that shares weights and biases among different GUs. Compared to standard FC layers/linear layers of equal scale, these modules have fewer parameters, thereby reducing the solution space for the optimal policy. However, despite its advantage, parameter sharing limits model expressiveness. Inspired by [31], HEP2Net addresses this issue by incorporating hypernetworks to generate network parameters for different UAVs/GUs.

2) *Hypernetwork-Based EP2Net*: Hypernetworks are a type of neural networks that generate parameters for another neural network [43]. As shown in Fig. 6, we employ a shared hypernetwork, denoted as $HN_{in}^{uav}(\cdot)$, to generate parameters of $LN_{in}^{uav}(\cdot)$ for each UAV- n' by taking its feature as input. As a result, the parameters of $LN_{in}^{uav}(\cdot)$ vary across different UAVs and different time slots. Accordingly, the formulation of $f_{in}^{uav}(\cdot)$ becomes

$$\begin{aligned} f_{in}^{uav}(\tilde{\mathbf{x}}_t^{n,uav,n'}) = & \mathbf{LN}_{in}^{uav} \left(\tilde{\mathbf{x}}_t^{n,uav,n'}; \right. \\ & \left. \mathbf{HN}_{in}^{uav}(\tilde{\mathbf{x}}_t^{n,uav,n'}; \varrho_{in}^{uav}) \right). \end{aligned} \quad (53)$$

Here, the hypernetwork $HN_{in}^{uav}(\cdot)$ is constructed as an FC layer, and ϱ_{in}^{uav} denote its learnable parameters. The network $LN_{in}^{user}(\cdot)$ for GUs is modified in the same manner with $LN_{in}^{uav}(\cdot)$. In the output module, we also construct a shared hypernetwork, denoted as $HN_{out}^{user}(\cdot)$, to generate parameters of $LN_{out}^{user}(\cdot)$ by taking each GU- m 's features as input. As $\tilde{\mathbf{x}}_t^{n,user,m}$ is fed into the hypernetwork, it is no longer necessary to be a direct input of $LN_{out}^{user}(\cdot)$. Consequently, $f_{out}^{user}(\cdot)$ can be represented as

$$\begin{aligned} f_{out}^{user}(\mathbf{h}_t^n, \tilde{\mathbf{x}}_t^{n,user,m}) = & \mathbf{LN}_{out}^{user} \left(\mathbf{h}_t^n; \right. \\ & \left. \mathbf{HN}_{out}^{user}(\tilde{\mathbf{x}}_t^{n,user,m}; \varrho_{out}^{user}) \right), \end{aligned} \quad (54)$$

where ϱ_{out}^{user} represent the learnable parameters of $HN_{out}^{user}(\cdot)$. The other part of HEP2Net remains consistent with FEP2Net.

In summary, HEP2Net introduces hypernetworks to generate network parameters, resulting in distinct network parameters for different entities across different time slots. This approach

can be seen as a compromise between the parameter sharing of FEP2Net and no parameter sharing of traditional policy networks. As a result, HEP2Net enhances model expressiveness, while still maintaining the EP2Net structure.

3) *Computational Complexity*: Since EP2Nets only improve the input module and output module, we focus on the computational complexity of these two components. In the case of HEP2Net, both its input module and output module encompass two key processes: the forward computation involving an FC layer/linear layer and the parameter generation via hypernetworks. The computational complexity of the first process mirrors that of a standard FC layer/linear layer and can be calculated as $\mathcal{O}(d_o d_z)$ for the input module and $\mathcal{O}(4d_h(M+1))$ for the output module. Here, d_o , d_z and d_h denote the dimensions of the observation vector, embedding feature, and hidden state, respectively, while $4(M+1)$ is the total number of discrete actions. Concerning the parameter generation process, a hypernetwork comprising a linear layer computes network parameters for all UAVs/GUs. Its computational complexity is $\mathcal{O}((N-1)d_{\text{uav}}d_z(d_{\text{uav}}+1) + Md_{\text{user}}d_z(d_{\text{user}}+1))$ for input module and $\mathcal{O}(4Md_{\text{user}}(d_{\text{user}}+1))$ for the output module. Here, $d_z(d_{\text{uav}}+1)$, $d_z(d_{\text{user}}+1)$, and $4(d_{\text{user}}+1)$ represent the number of parameters that the hypernetworks need to generate, corresponding to their output dimension. Compared to FEP2Net or traditional policy networks consisting of FC layers and linear layers, the incorporation of hypernetworks adds computational complexity to HEP2Net. Nevertheless, the parameter generation process can be executed in parallel. When fully parallelized, the extra computational complexity remains a constant value independent of the number of UAVs and GUs, thereby rendering it acceptable.

C. Data Augmentation-Based Learning Process

In this section, we incorporate data augmentation into the learning process by exploiting rotational and reflection symmetries. For more details about rotational and reflection symmetries, see Section IV-D2.

Since the rotation and reflection group G^{rt} is a symmetry group of our formulated Dec-POMDP, the state-observation-action tuples defined by the transformation $(L_g^{\text{rt}}, U_g^{\text{rt}}, k_g^{\text{rt}})$ share the same reward and state transition probability. Therefore, we can generate synthetic experience samples by directly applying symmetric rotations and reflections to each real experience sample.

As shown in Fig. 4, during the learning process, all UAV agents interact with the environment by selecting actions according to their EP2Net-based policies. The experience sample generated at each time slot is represented as $(s_t, \{\mathbf{o}_t^n\}_n, \{a_t^n\}_n, s_{t+1}, \{\mathbf{o}_{t+1}^n\}_n, r_t)$. We create an augmented experience sample set by applying symmetric rotations and reflections to each experience sample:

$$e_t^{\text{aug}} = \left\{ \left(L_g^{\text{rt}}[s_t], \{U_g^{\text{rt}}[\mathbf{o}_t^n]\}_n, \{k_g^{\text{rt}}[a_t^n]\}_n, L_g^{\text{rt}}[s_{t+1}], \{U_g^{\text{rt}}[\mathbf{o}_{t+1}^n]\}_n, r_t \right) \mid g \in G^{\text{rt}} \right\}, \quad (55)$$

Algorithm 1: Data Augmentation-Based Learning Process:

Input: initial parameters of EP2Nets, denoted as θ , and empty replay buffer $\mathcal{D}^{\text{aug}}$

- 1: **repeat**
- 2: Reset environment state
- 3: **for** $t = 1, \dots, T$ **do**
- 4: Obtain the environment state s_t
- 5: **for** all agent $n \in \mathcal{N}$ **do**
- 6: Observe $\mathbf{o}_t^n$ and select action a_t^n according to action probabilities generated by EP2Net
- 7: Execute a_t^n in the environment
- 8: **end for**
- 9: **for** all agent $n \in \mathcal{N}$ **do**
- 10: Obtain the next observation $\mathbf{o}_{t+1}^n$
- 11: **end for**
- 12: Obtain the next state s_{t+1} and team reward r_t
- 13: Generate augmented experience sets according to (55), and store them into $\mathcal{D}^{\text{aug}}$.
- 14: **end for**
- 15: **if** it's time to update **then**
- 16: Randomly sample a batch of transitions $\mathcal{B} = \{(s_t, \{\mathbf{o}_t^n\}_n, \{a_t^n\}_n, s_{t+1}, \{\mathbf{o}_{t+1}^n\}_n, r_t)\}$ from $\mathcal{D}^{\text{aug}}$
- 17: Update θ using arbitrary MADRL algorithm
- 18: **end if**
- 19: **until** Convergence

Output: well-trained EP2Net parameters θ^*

These experience samples are stored in an augmented replay buffer $\mathcal{D}^{\text{aug}}$. Periodically, we take a batch of experience samples from $\mathcal{D}^{\text{aug}}$ to train policies using a specific MADRL algorithm. The complete training procedure is outlined in Algorithm 1. After achieving convergence, the well-trained EP2Net model can be deployed on each UAV, enabling decentralized and real-time decision making.

In traditional MADRL algorithms, the replay buffer only contains true experience samples gathered through the interactions between agents and the environment. In contrast, the proposed augmented replay buffer stores both true experience samples and synthetic samples generated by rotating and reflecting true samples. Transforming existing samples is much simpler than implementing agents-environment interactions, especially in real-world training. Therefore, this data augmentation method is equivalent to expanding true experience samples by $|G^{\text{rt}}| = 8$, significantly enhancing their utility efficiency. It is noteworthy that the proposed data augmentation-based learning process is also algorithm-agnostic and can be applied to most MADRL algorithms.

D. SymmQMIX Algorithm

The EP2Net network structure and data augmentation methods do not depend on any specific learning algorithm and can be combined with most MADRL algorithms. The QMIX algorithm [12] has demonstrated state-of-the-art performance in various MADRL tasks. In this section, we incorporate the

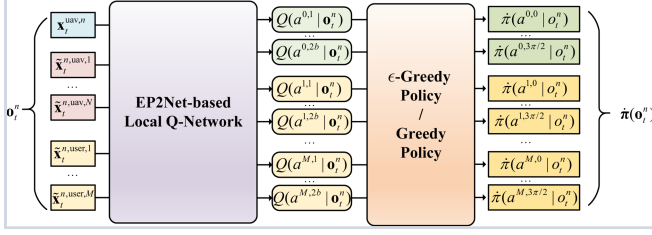


Fig. 7. Illustrations of EP2Net-based local Q-networks in SymmQMIX. Each EP2Net computes individual Q-values for discrete actions. These values are then utilized to produce action probabilities according to an ϵ -greedy policy or a greedy policy.

aforementioned symmetry methods into QMIX, resulting in the SymmQMIX algorithm.

QMIX learns a joint Q-value $Q(s_t, \mathbf{a}_t)$, and factors it into a set of individual Q-values for each agent, $\{Q(o_t^n, a_t^n)\}_{n \in \mathcal{N}}$. The joint Q-value represents the expected cumulative reward starting from state s_t , all agents taking the joint action $\mathbf{a}_t$, and thereafter following their respective optimal policies. Differently, the individual Q-value represents the contribution of each agent. To achieve this, QMIX comprises a set of local Q-networks for estimating the individual Q-values, and a mixing network that combines all individual Q-values into a joint Q-value. QMIX adopts a framework of centralized training with decentralized execution. During training, each agent selects actions based on their individual Q-values using an ϵ -greedy policy:

$$\hat{\pi}(a_t^n | o_t^n) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|}, & \text{if } a_t^n = \arg \max_a Q(o_t^n, a) \\ \frac{\epsilon}{|\mathcal{A}|}, & \text{otherwise.} \end{cases} \quad (56)$$

The training loss for QMIX is formulated using the Bellman equation for the joint Q-value:

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathcal{D}} \left[\left(r_t + \gamma \max_{\mathbf{a}_{t+1}} Q_{\theta^-}(s_{t+1}, \mathbf{a}_{t+1}) - Q_{\theta}(s_t, \mathbf{a}_t) \right)^2 \right]. \quad (57)$$

Here, $\mathcal{D}$ represents the experience replay buffer, θ denote all learnable parameters in QMIX, and θ^- are periodically copied from θ . After convergence, each agent selects actions with the highest Q-values:

$$\hat{\pi}(a_t^n | o_t^n) = \begin{cases} 1, & \text{if } a_t^n = \arg \max_a Q(o_t^n, a) \\ 0, & \text{otherwise.} \end{cases} \quad (58)$$

This approach facilitates decentralized and real-time execution.

In SymmQMIX, the local Q-networks are replaced with any implementation of EP2Nets, such as FEP2Nets or HEP2Nets. Only the input and output modules of QMIX are modified, keeping the backbone middle module remains unchanged. While EP2Nets are designed for generating action probabilities, their structures are also applicable to produce individual Q-values. As illustrated in Fig. 7, we configure each EP2Net to output individual Q-values for discrete actions. Then, the action probabilities can be obtained according to greedy or ϵ -greedy policies. During the training process, we employ data augmentation to augment experience samples as described in Section V-C. As a result, the experience replay buffer $\mathcal{D}$ is replaced with the augmented

replay buffer $\mathcal{D}^{\text{aug}}$. All other aspects remain consistent with the original QMIX algorithm.

VI. SIMULATION RESULTS

In this section, we first validate the effectiveness of the proposed approach by comparing SymmQMIX with benchmark algorithms. Then, we conduct ablation studies to investigate the impact of different components in SymmQMIX. Finally, we verify the scalability of the proposed approach across various population sizes.

A. Experiment Setup

1) *Environment Parameters*: The service region of UAVs is restricted to a square area of $2000 \text{ m} \times 2000 \text{ m}$. The service duration is 1500 s, divided into 150 time slots. During the learning process, each training episode corresponds to the entire service duration. At the beginning of each episode, we employ certain rules to randomly generate four non-overlapping hotspots. Within each hotspot, an equal number of GUs are generated. The position of each GU remains unchanged throughout an episode. UAVs are randomly positioned within the whole area, maintaining a constant flying altitude of $H = 100 \text{ m}$ and a fixed speed of $V = 10 \text{ m/s}$. For the channel model, we employ parameters corresponding to the urban scenario [11]: $\alpha = 2$, $a = 9.61$, $b = 0.15$, $\eta^{\text{LoS}} = 1 \text{ dB}$, $\eta^{\text{NLoS}} = 20 \text{ dB}$, and $f^c = 2 \text{ GHz}$. The transmit power of UAVs is 0.5 W , while the bandwidth is set as 1 MHz . The PSD of AWGN is -169 dBm/Hz . The communication range and sensing range of UAVs are defined as 200 m and 500 m , respectively. Unless otherwise stated, we set the numbers of UAVs and GUs as 4 and 64 (16 in each hotspot). In Section VI-D, we will adjust the population size to evaluate the scalability of the proposed method.

2) *Algorithm Implementations*: Regarding the SymmQMIX algorithm, the local Q network defaults to the HEP2Net structure. In Section VI-C, we will further compare the performance of FEP2Net and HEP2Net. For clarity, the algorithms corresponding to these two structures are named SymmQMIX-F and SymmQMIX-H respectively. QMIX serves as the foundational algorithms for all benchmark algorithms. All code and hyperparameters are based on the optimized QMIX implementation from project PyMARL2.

B. Performance Evaluation of the Proposed Approach

1) *Performance Comparison With MADRL Algorithms*: In this subsection, we evaluate the performance improvement introduced by the proposed symmetry-augmented MADRL approach when combined with QMIX. We employ the original QMIX algorithm and its variants enhanced by three permutation-invariant networks as benchmarks.

- *QMIX*: The original QMIX algorithm [12]. Both the input and output modules of local Q-networks comprise an FC layer, while the middle module consists of a gated recurrent unit (GRU).
- *SymmQMIX-H*: As described in Section V-D, SymmQMIX is a combination of QMIX and the symmetry-augmented

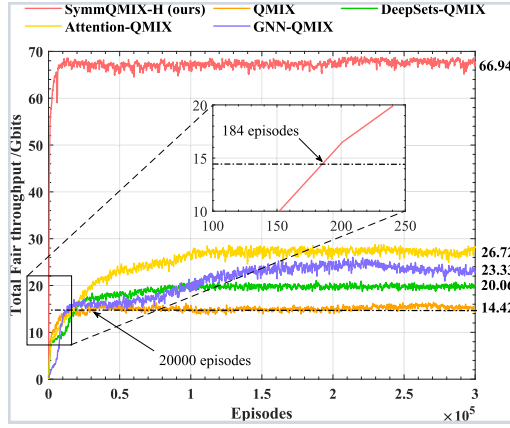


Fig. 8. Learning curves of SymmQMIX and benchmark algorithms for 200000 episodes. The numbers of UAVs and GUs are set as 4 and 64.

MADRL approach. In SymmQMIX-H, the local Q-networks in QMIX are replaced with HEP2Nets.

- *DeepSets-QMIX*: Following [30], DeepSets-QMIX incorporates DeepSets [38] into the input modules of local Q-networks. Specifically, DeepSets applies a shared FC layer to processes each UAV/GU feature individually, followed by aggregation through summation.
- *Attention-QMIX*: Similar to [32], Attention-QMIX integrates a self-attention mechanism into the input modules of local Q-networks. It processes each entity feature separately and calculates an attention coefficient for each entity to facilitate weighted summation.
- *GNN-QMIX*: Similar to [11], GNN-QMIX incorporates GNN into the input modules of local Q-networks. Specifically, GNN learns a representation for each UAV/GU by aggregating embedding features from other entities. These representations are aggregated by sum pooling.

Fig. 8 illustrates the learning curves of these algorithms concerning the total fair throughput. SymmQMIX-H significantly outperforms all benchmark algorithms in terms of both sample efficiency and converged performance. Specifically, QMIX exhibits the poorest performance, converging at approximately 20000 episodes with an total fair throughput of only 14.42 Gbits. In contrast, SymmQMIX-H achieves this level of performance in only 184 episodes, yielding a remarkable 109-fold improvement in sample efficiency. Furthermore, after convergence, SymmQMIX-H achieves total fair throughput of 66.94 Gbits, representing a 4.6-fold improvement over QMIX. This notable performance improvement can be primarily attributed to SymmQMIX-H's utilization of symmetries. QMIX neglects symmetries, leading to an expansive large state-action space that requires a considerable number of experience samples for training and is susceptible to local optima. SymmQMIX-H introduces HEP2Net on top of QMIX, effectively leveraging permutation symmetry to reduce the state-action space, and further enhance sample efficiency through data augmentation. Furthermore, the reduction in the state-action space prevents the algorithm from falling into local optima, resulting in improved converged performance.

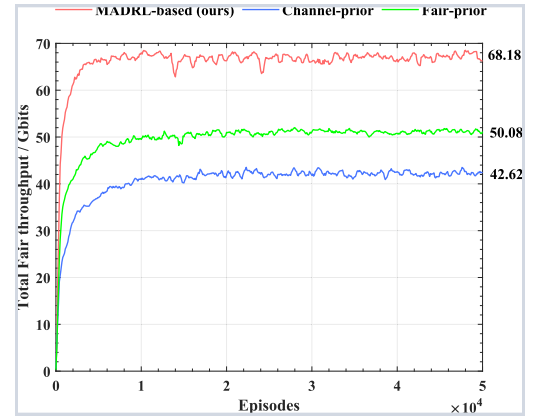


Fig. 9. Learning curves of three user scheduling methods, all learned by SymmQMIX-H. The numbers of UAVs and GUs are set as 4 and 64.

DeepSets-QMIX, Attention-QMIX, and GNN-QMIX bring performance improvements of 39.11%, 85%, and 61.6% over QMIX, respectively. However, all of them fall significantly short of SymmQMIX's performance. These algorithms utilize natural permutation-invariant networks to ensure that changes in the entity order in the states and observations do not alter the policy's output. These networks utilize a shared network to individually process entity features and then aggregate them in different ways. The permutation invariance property proves effective in tasks where actions are independent of entity order, such as pure UAV trajectory design, as it reduces redundancy in the state space due to entity permutations. Nevertheless, in the joint UAV trajectory design and user scheduling problem, changes in entity order simultaneously affect state and action permutations, leading to transformations in policy outputs. This permutation equivariance property cannot be achieved using DeepSets, Attention, and GNN. In contrast, our proposed EP2Net achieves it by establishing connections between observations and actions, leading to reduced state-action space and improved performance. Moreover, beyond permutation symmetry, SymmQMIX-H also accounts for rotational and reflection symmetries, further enhancing algorithm performance.

2) *Performance Comparison With Heuristic Algorithms*: In this subsection, we design two benchmark algorithms to validate the effectiveness of utilizing MADRL for learning user scheduling. These benchmark algorithms utilize MADRL solely for UAV trajectory design, while user scheduling decisions are made based on the following schemes:

- *Channel-prior*: UAVs select the GU with the best channel among the associated GUs to provide service.
- *Fair-prior*: UAVs choose the GU with the lowest average throughput among the associated GUs for service.

We denote the proposed method as “MADRL-based”. All three methods employ SymmQMIX-H for learning UAV policies. Learning curves can be observed in Fig. 9. Table II presents the average throughput and fairness index achieved by these algorithms after convergence.

As shown in Table II, the channel-prior method prioritizes higher throughput at the expense of fairness, while the fair-prior method sacrifices system throughput for improved user fairness.

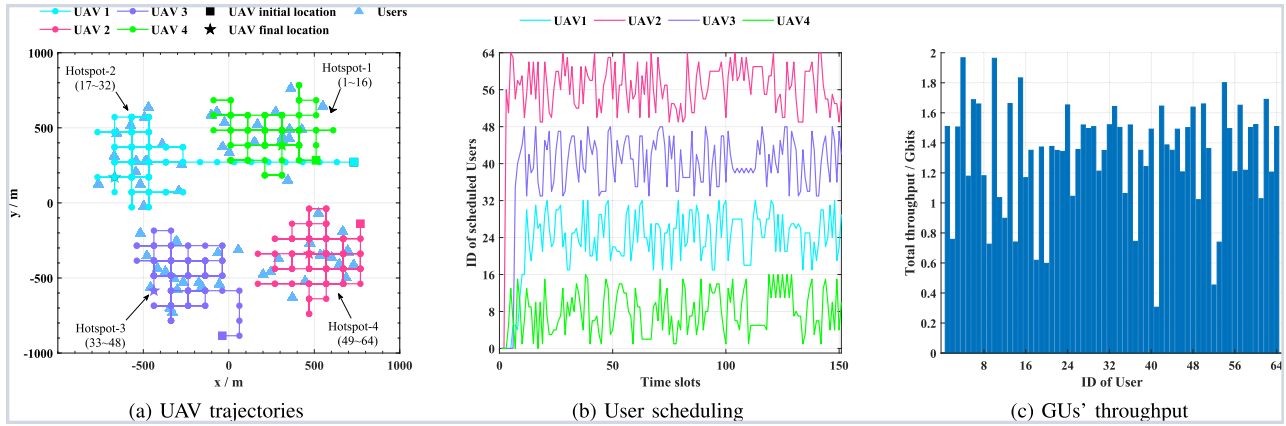


Fig. 10. Illustrative policies: a case study of 4 UAVs and 64 GUs. (a) Four UAVs autonomously move to various hotspot areas to establish communication coverage. (b) UAV-1, UAV-2, UAV-3, and UAV-4 correspond to hotspot areas-2, hotspot areas-4, hotspot areas-3, and hotspot areas-1, respectively. They sequentially serve GUs in their respective areas to ensure equitable coverage. (c) The throughput of all GUs exceeds 0, indicating that each GU has access to communication services. The throughput among GUs is relatively uniform, ensuring a certain level of fairness.

TABLE II
COMPARISON OF FAIR THROUGHPUT, THROUGHPUT, FAIRNESS INDEX
AFTER CONVERGENCE

Metrics	MADRL-based	Channel-prior	Fair-prior
Total Fair throughput (Gbits)	68.18	42.62	50.08
Total Throughput (Gbits)	83.45	73.55	62.34
Fairness index	0.806	0.567	0.8

The bolded data indicates that these are the results produced by our algorithm.

The total fair throughput achieved by these two algorithms is 42.62 Gbits and 50.08 Gbits, respectively. In contrast, the proposed method outperforms in all metrics, with total throughput being 13.5% higher than channel-prior and fairness metrics also surpassing fairness-prior. It ultimately converges to total fair throughput of 68.18 Gbits, marking a 60 % and 36 % improvement over the channel-prior and fairness-prior scheduling methods, respectively. These results demonstrate the effectiveness of MADRL in learning user scheduling policies that strike a balance between enhancing system throughput and ensuring user fairness. These policies excel across all metrics, highlighting the superiority of MADRL over heuristic approaches.

Fig. 10 present the trajectory design and user scheduling policies learned by UAV agents in an example episode. As illustrated in Fig. 10(a), UAVs cooperatively provide communication services across the entire area. Specifically, four UAVs navigate to distinct hotspot areas, maintaining significant distances between each other to reduce cross-link interference. Each individual UAV patrols within its corresponding region to ensure service coverage for all GUs. Fig. 10(b) presents the outcomes of user scheduling, with GUs categorized into groups labeled 1 ~ 16, 17 ~ 32, 33 ~ 48, and 49 ~ 64, corresponding to the four hotspot areas. Each UAV sequentially serves GUs within its corresponding hotspot area, ensuring equitable and adequate service. The total throughput achieved by each GU are illustrated in Fig. 10(c), demonstrating that every GU receives service. These findings underscore the optimization of system throughput and fairness among GUs.

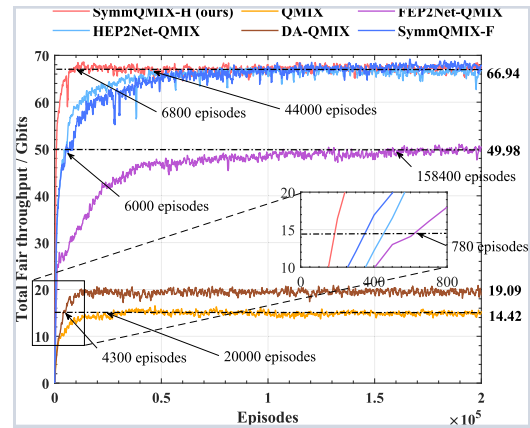


Fig. 11. Learning curves of ablation experiments for 200,000 episodes. The numbers of UAVs and GUs are set as 4 and 64.

C. Ablation Studies

In this section, we conduct ablation experiments to assess the contributions of various components within the proposed symmetry-augmented approach. The algorithms under consideration are listed as follows:

- *FEP2Net-QMIX* To isolate the effect of EP2Net, we implement the simplest EP2Net variant, i.e., FEP2Nets, to replace the local Q-networks in QMIX.
- *HEP2Net-QMIX*: To investigate the performance improvement brought about by hypernetworks, we replace the FEP2Nets in FEP2Net-QMIX with HEP2Nets.
- *DA-QMIX*: To examine the impact of data augmentation, we introduce data augmentation alone into QMIX.
- *SymmQMIX-F*: To further investigate the impact of data augmentation, we introduce data augmentation into FEP2Net-QMIX. The only difference between SymmQMIX-F and SymmQMIX-H is the replacement of HEP2Nets with FEP2Nets.

The corresponding learning curves are displayed in Fig. 11.

1) *Performance Improvement of EP2Nets*: FEP2Net-QMIX achieves the converged performance of QMIX with just 780 episodes, marking an impressive 25.6-fold improvement in sample efficiency. Ultimately, it converges around 40600 episode, reaching total fair throughput of 47.85 Gbits, which is 3.3 times higher than QMIX. These results indicate that introducing the simplest form of EP2Net alone can lead to a substantial performance enhancement.

2) *Performance Improvement of Hypernetworks*: HEP2Net introduces hypernetworks into FEP2Net to generate network parameters for different entities, addressing the limitations in representational capacity caused by parameter sharing. This approach further enhances both sample efficiency and converged performance. Specifically, HEP2Net-QMIX attains the converged performance of FEP2Net-QMIX at 4200 episodes, resulting in an 11-fold improvement in sample efficiency. HEP2Net-QMIX reaches convergence at around 44000 episodes, achieving performance on par with SymmQMIX-H and representing a 42.6% improvement over FEP2Net-QMIX.

3) *Performance Improvement of Data Augmentation*: SymmQMIX-H introduces data augmentation into HEP2Net-QMIX. It converges in just 6800 episodes, resulting in a 6.5-fold improvement in sample efficiency. Additionally, DA-QMIX, which introduces data augmentation into QMIX, achieves a 4.7-fold improvement in sample efficiency and a 32.4% converged performance enhancement. SymmQMIX-F, which integrates data augmentation into FEP2Net-QMIX, results in a 10.6-fold improvement in sample efficiency and achieves converged performance comparable to SymmQMIX.

4) *Analysis*: The above results demonstrate that EP2Nets, hypernetworks, and data augmentation contribute to varying degrees of performance improvement in SymmQMIX. Specifically, EP2Net's primary role lies in reducing the state-action space by introducing permutation symmetry. In our simulation scenario involving 4 UAVs and 64 GUs, there are a total of $4! \times 64! = 3 \times 10^{90}$ permutations. EP2Net, due to its permutation equivariance, learns the optimal policy for all permutations with just one experience sample. This effectively reduces the state-action space by a factor of $\frac{1}{3 \times 10^{90}}$. Data augmentation further enhances sample efficiency by augmenting each experience sample 8-fold through symmetric rotations and reflections. Hypernetworks play a role to enhance model expressiveness. The most substantial performance improvement comes from permutation symmetry, as the redundancy resulting from entity permutations far outweighs that from rotations and reflections. Introducing hypernetworks or data augmentation separately based on EP2Nets can achieve converged performance comparable to SymmQMIX-H. However, the highest sample efficiency is achieved when all three components are combined.

D. Performance Comparison Across Different Population Sizes

In this section, we further validate the scalability of the proposed symmetry-based MADRL approach by comparing **SymmQMIX-H** and **QMIX** under various population sizes. Additionally, to verify the population invariance of EP2Nets, we

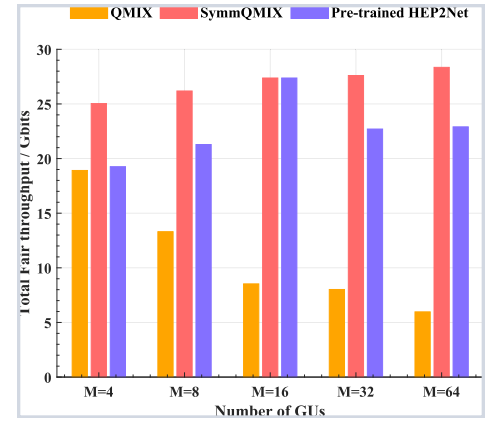


Fig. 12. Comparison of the converged fair throughput with respect to the number of GUs. The number of UAVs is set as 2. The pre-trained model was trained in the scenario with 16 GUs.

also evaluate the performance of pre-trained HEP2Nets directly applied to scenarios with different population sizes. The specific configurations are as follows:

- *Pre-trained HEP2Net*: We apply HPE2Nets trained in a scenario with a specific population size to scenarios with varying population sizes. Specifically, for scenarios involving different numbers of GUs, we utilize the model trained with 16 GUs as the pre-trained model. For scenarios involving different numbers of UAVs, the model trained with 6 UAVs are used as the pre-trained model.

First, we fix the number of UAVs at 2, and vary GU count from 4 to 64. Fig. 12 illustrates the performance comparison in terms of total fair throughput after convergence. Notably, SymmQMIX-H consistently outperforms the QMIX algorithm across different GU counts, with this performance advantage increasing as the number of GUs rises. Specifically, as the number of GUs increases from 4 to 64, the fair throughput of SymmQMIX-H steadily ascends from 25.03 Gbits to 28.35 Gbits. In contrast, this metric for QMIX drops from 19 Gbits to 6 Gbits. Consequently, the performance advantage of SymmQMIX-H relative to QMIX increases from 33.8% to a 4.2-fold improvement. The performance degradation of QMIX can be primarily attributed to the explosive state-action space as the number of GUs increases. SymmQMIX-H mitigates this issue by exploiting symmetries, enabling it learn effective policies even in scenarios with a large number of GUs. These results highlight the superior scalability of SymmQMIX-H.

Next, we keep the GU count at 64 while varying the number of UAVs from 2 to 10. The performance comparison is depicted in Fig. 13. With an increase in UAVs, the fair throughput of all algorithms increases with the rising number of UAVs. However, as the number of UAVs increases, the state space expands exponentially, and the cooperation among UAV agents becomes more complex. This complexity makes it challenging for QMIX to learn effective policies. Therefore, when the number of UAVs increases from 2 to 10, the total fair throughput of QMIX only increases from 6 Gbits to 38.7 Gbits. In contrast, the total fair throughput of SymmQMIX-H surges from 28.64 Gbits to

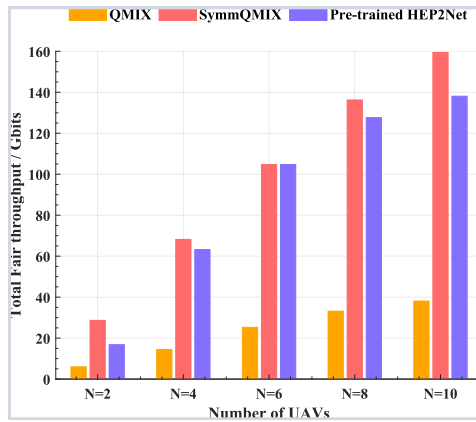


Fig. 13. Comparison of the converged fair throughput with respect to the number of UAVs. The number of GUs is set as 64. The pre-trained model was trained in the scenario with 6 UAVs.

159.4 Gbps. The performance gap remains stable at around 4.5, further emphasizing the scalability of the proposed approach.

Finally, we observe that the pretrained HEP2Net models achieve satisfactory performance in scenarios with different numbers of GUs and UAVs. For instance, with 2 UAVs and 32 GUs, the total fair throughput of the pretrained HEP2Net is only 17.8% lower than learning from scratch (i.e., SymmQMIX), surpassing QMIX by a factor of 2.8. In the scenarios involving 10 UAVs and 64 GUs, the total fair throughput of the pretrained HEP2Net is only 15.4% lower than learning from scratch, outperforming QMIX by twice the margin. While these pre-trained models may exhibit slight performance degradation compared to learning from scratch, they consistently outperform the QMIX algorithm, demonstrating the HEP2Net's ability to handle varying population sizes.

VII. CONCLUSION

In this paper, we present a novel symmetry-augmented MADRL approach to solve UAV trajectory design and user scheduling problems. Compared with existing MADRL-based methods, we uniquely incorporate symmetries into both the policy networks and the learning process. The proposed EP2Nets significantly reduce redundancy in the state-action space caused by different permutations. Moreover, the data augmentation-based learning process further enhances sample efficiency by exploiting rotational and reflection symmetries. Extensive simulation results demonstrate the substantial enhancement in both sample efficiency and converged performance achieved by integrating the proposed approach into QMIX algorithm, especially in scenarios involving large population sizes. In future research, we will explore more symmetries within the environment and address situations involving non-ideal symmetries.

REFERENCES

- [1] L. Gupta, R. Jain, and G. Vaszkun, "Survey of important issues in UAV communication networks," *IEEE Commun. Surveys Tut.*, vol. 18, no. 2, pp. 1123–1152, Second Quarter, 2016.
- [2] H. Wang et al., "Joint resource allocation on slot, space and power towards concurrent transmissions in UAV ad hoc networks," *IEEE Trans. Wireless Commun.*, vol. 21, no. 10, pp. 8698–8712, Oct. 2022.
- [3] H. Zhao, H. Wang, W. Wu, and J. Wei, "Deployment algorithms for UAV airborne networks toward on-demand coverage," *IEEE J. Select. Areas Commun.*, vol. 36, no. 9, pp. 2015–2031, Sep. 2018.
- [4] Y. Zeng, Q. Wu, and R. Zhang, "Accessing from the sky: A tutorial on UAV communications for 5G and beyond," in *Proc. IEEE*, vol. 107, no. 12, pp. 2327–2375, Dec. 2019.
- [5] Q. Wu, Y. Zeng, and R. Zhang, "Joint trajectory and communication design for Multi-UAV enabled wireless networks," *IEEE Trans. Wireless Commun.*, vol. 17, no. 3, pp. 2109–2121, Mar. 2018.
- [6] C. Zhan, Y. Zeng, and R. Zhang, "Energy-efficient data collection in UAV enabled wireless sensor network," *IEEE Wireless Commun. Lett.*, vol. 7, no. 3, pp. 328–331, Jun. 2018.
- [7] X. Zhang, J. Zhang, J. Xiong, L. Zhou, and J. Wei, "Energy-efficient Multi-UAV-Enabled multiaccess edge computing incorporating NOMA," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 5613–5627, Jun. 2020.
- [8] C. Shen, T.-H. Chang, J. Gong, Y. Zeng, and R. Zhang, "Multi-UAV interference coordination via joint trajectory and power control," *IEEE Trans. Signal Process.*, vol. 68, pp. 843–858, 2020.
- [9] L. Liu, S. Zhang, and R. Zhang, "CoMP in the sky: UAV placement and movement optimization for multi-user communications," *IEEE Trans. Commun.*, vol. 67, no. 8, pp. 5645–5658, Aug. 2019.
- [10] X. Zhou, X. Zhang, H. Zhao, J. Xiong, and J. Wei, "Constrained soft actor-critic for energy-aware trajectory design in UAV-Aided IoT networks," *IEEE Wireless Commun. Lett.*, vol. 11, no. 7, pp. 1414–1418, Jul. 2022.
- [11] X. Zhang, H. Zhao, J. Wei, C. Yan, J. Xiong, and X. Liu, "Cooperative trajectory design of multiple UAV base stations with heterogeneous graph neural networks," *IEEE Trans. Wireless Commun.*, vol. 22, no. 3, pp. 1495–1509, Mar. 2023.
- [12] T. Rashid, M. Samvelyan, C. Schroeder, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4295–4304.
- [13] P. Sunehag et al., "Value-decomposition networks for cooperative multi-agent learning," in *Proc. 17th Int. Conf. Auton. Agents Multiagent Syst.*, 2018, pp. 2085–2087.
- [14] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6379–6390.
- [15] L. Yuan, Z. Zhang, L. Li, C. Guan, and Y. Yu, "A survey of progress on cooperative multi-agent reinforcement learning in open environment," 2023, *arXiv:2312.01058*.
- [16] C. Yan et al., "Collision-avoiding flocking with multiple fixed-wing UAVs in obstacle-cluttered environments: A task-specific curriculum-based MADRL approach," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 8, pp. 10894–10908, Aug. 2024.
- [17] Q. Wang, W. Zhang, Y. Liu, and Y. Liu, "Multi-UAV dynamic wireless networking with deep reinforcement learning," *IEEE Commun. Lett.*, vol. 23, no. 12, pp. 2243–2246, Dec. 2019.
- [18] W. Zhang, Q. Wang, X. Liu, Y. Liu, and Y. Chen, "Three-dimension trajectory design for Multi-UAV wireless network with deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 70, no. 1, pp. 600–612, Jan. 2021.
- [19] Y. Yuan, L. Lei, T. X. Vu, S. Chatzinotas, S. Sun, and B. Ottersten, "Energy minimization in UAV-Aided networks: Actor-critic learning for constrained scheduling optimization," *IEEE Trans. Veh. Technol.*, vol. 70, no. 5, pp. 5028–5042, May 2021.
- [20] R. Ding, Y. Xu, F. Gao, and X. Shen, "Trajectory design and access control for air-ground coordinated communications system with multi-agent deep reinforcement learning," *IEEE Internet Things J.*, vol. 9, no. 8, pp. 5785–5798, Apr. 2022.
- [21] X. Zhou et al., "Joint UAV trajectory and communication design with heterogeneous multi-agent reinforcement learning," *Sci. China Inf. Sci.*, vol. 67, no. 3, pp. 1–21, 2024.
- [22] Q. Long, Z. Zhou, A. Gupta, F. Fang, Y. Wu, and X. Wang, "Evolutionary population curriculum for scaling multi-agent reinforcement learning," in *Proc. Int. Conf. Learn. Representations*, 2020.
- [23] M. Zinkevich and T. R. Balch, "Symmetry in Markov decision processes and its implications for single agent and multiagent learning," in *Proc. 18th Int. Conf. Mach. Learn.*, 2001, Art. no. 632.
- [24] C. Clark and A. Storkey, "Teaching deep convolutional neural networks to play go," in *Proc. 32nd Int. Conf. Mach. Learn.*, 2015, pp. 1766–1774.

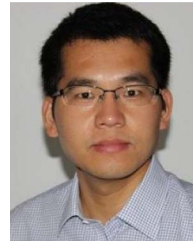
- [25] G. N. C. Simm, R. Pinsler, G. Csányi, and J. M. Hernández-Lobato, "Symmetry-aware actor-critic for 3D molecular design," in *Proc. Int. Conf. Learn. Representations*, 2021.
- [26] E. van der Pol et al., "MDP homomorphic networks: Group symmetries in reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 4199–4210.
- [27] Y. Lin, J. Huang, M. Zimmer, Y. Guan, J. Rojas, and P. Weng, "Invariant transform experience replay: Data augmentation for deep reinforcement learning," *IEEE Robot. Autom. Lett.*, vol. 5, no. 4, pp. 6615–6622, Oct. 2020.
- [28] E. van der Pol, H. van Hoof, F. A. Oliehoek, and M. Welling, "Multi-agent MDP homomorphic networks," in *Proc. Int. Conf. Learn. Representations*, 2022.
- [29] W. Wang et al., "From few to more: Large-scale dynamic multiagent curriculum learning," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 7293–7300.
- [30] Y. Li et al., "Permutation invariant policy optimization for mean-field multi-agent reinforcement learning: A principled approach," 2021, *arXiv:2105.08268*.
- [31] J. Hao et al., "Boosting multi-agent reinforcement learning via permutation invariant and permutation equivariant networks," in *Proc. Int. Conf. Learn. Representations*, 2022.
- [32] C. Yan et al., "PASCAL: PopULAtion-Specific curriculum-based MADRL for collision-free flocking with large-scale fixed-wing UAV swarms," *Aerosp. Sci. Technol.*, vol. 133, 2023, Art. no. 108091.
- [33] Y. Xu and W. Yin, "A block coordinate descent method for regularized multiconvex optimization with applications to nonnegativetensor factorization and completion," *SIAM J. Imag. Sci.*, vol. 6, pp. 1758–1789, 2013.
- [34] B. R. Marks and G. P. Wright, "A general inner approximation algorithm for nonconvex mathematical programs," *Operations Res.*, vol. 26, no. 4, pp. 681–683, 1978.
- [35] J. K. Gupta, M. Egorov, and M. Kochenderfer, "Cooperative multi-agent control using deep reinforcement learning," in *Proc. Auton. Agents Multi-agent Syst.*, 2017, pp. 66–83.
- [36] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.
- [37] S. Yin and F. R. Yu, "Resource allocation and trajectory design in UAV-aided cellular networks based on multiagent reinforcement learning," *IEEE Internet Things J.*, vol. 9, no. 4, pp. 2933–2943, Feb. 2022.
- [38] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. R. Salakhutdinov, and A. J. Smola, "Deep sets," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 3391–3401.
- [39] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
- [40] P. W. Battaglia et al., "Relational inductive biases, deep learning, and graph networks," 2018, *arXiv: 1806.01261*.
- [41] R. Zhong, X. Liu, Y. Liu, and Y. Chen, "Multi-agent reinforcement learning in NOMA-aided UAV networks for cellular offloading," *IEEE Trans. Wireless Commun.*, vol. 21, no. 3, pp. 1498–1512, Mar. 2022.
- [42] W. R. Scott, *Group Theory*. North Chelmsford, MA, USA: Courier Corporation, 2012.
- [43] D. Ha, A. Dai, and Q. V. Le, "HyperNetworks," in *Proc. Int. Conf. Learn. Representations*, 2017.



Xuanhan Zhou received the BE and PhD degrees from National University of Defense Technology (NUDT), China, in 2019 and 2024, respectively. Her research interests include UAV-assisted communication system and reinforcement learning.



Jun Xiong received the BS and PhD degrees from the National University of Defense Technology (NUDT), China, in 2009 and 2014, respectively. He is currently an associate professor with the College of Electronic Science and Technology, NUDT. His research interests include AI for communications and networking, physical layer security, and cognitive radio networks, where he has published more than 70 refereed articles.



Haitao Zhao (Senior Member, IEEE) received the BE, MSc and PhD degrees from the National University of Defense Technology (NUDT), China, in 2002, 2004 and 2009 respectively. And he is currently a professor with the Department of Cognitive Communications, College of Electronic Science and Technology at NUDT. Prior to this, he visited the Institute of ECIT, Queen's University of Belfast, U.K. and Hong Kong Baptist University. His main research interests include wireless communications, cognitive radio networks and self-organized networks, where he has published more than 100 refereed papers. He has served as a TPC member of IEEE ICC' 14-23, Globecom' 16-23, and guest editor for IEEE Communications Magazine.



Chao Yan received the BE degree in electrical engineering and automation from the China University of Mining and Technology, Xuzhou, China, in 2017, and the MS and PhD degrees in control science and engineering from the National University of Defense Technology, Changsha, China, in 2019, and 2023, respectively. He was a visiting PhD degree with the School of Mechanical and Aerospace Engineering, Nanyang Technological University, Singapore, from 2021 to 2022. He is currently an associate professor with the College of Automation Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing, China. His research interests include deep reinforcement learning, coordination control of UAV swarms.



Jibo Wei (Member, IEEE) received the BS and MS degrees in electronic engineering from the National University of Defense Technology (NUDT), Changsha, China, in 1989 and 1992, respectively, and the PhD in electronic engineering degree from Southeast University, Nanjing, China, in 1998. He is currently a professor with the Department of Cognitive Communications at NUDT. His research interests include Software Defined Radios (SDR), wireless network protocol and signal processing in communications, cooperative communication, and cognitive network, where he has published more than 200 refereed papers. He is a member of IEEE VTS. He is a council member of China Institute of Communications and an editorial committee member of the Journal on Communications.